

Sistemi operativi 2

INPUT OUTPUT

(parte 1)

Marco Guazzone

Università del Piemonte Orientale

marco.guazzone@uniupo.it

Sottosistema I/O

- L'esecuzione delle applicazioni utente non si basa solo sulla CPU e sulla memoria, ma anche su molti tipi diversi di componenti HW
- **Dispositivi I/O** sono componenti HW specificamente finalizzati a fornire input o dirigere output
 - Tastiere, mouse, schede di interfaccia di rete, dischi rigidi, ...

Sottosistema I/O (continua)

- **ESEMPIO:** Dispositivi di input e di output
 - Per ciascun dispositivo, indicare se viene generalmente utilizzato per l'ingresso (I), l'uscita (O) o entrambi (B)
 - [] Tastiera
 - [] Altoparlante
 - [] Schermo
 - [] Disco rigido
 - [] Microfono
 - [] Scheda di interfaccia di rete (NIC)
 - [] Schede di memoria (ovvero schede flash)

Sottosistema I/O (continua)

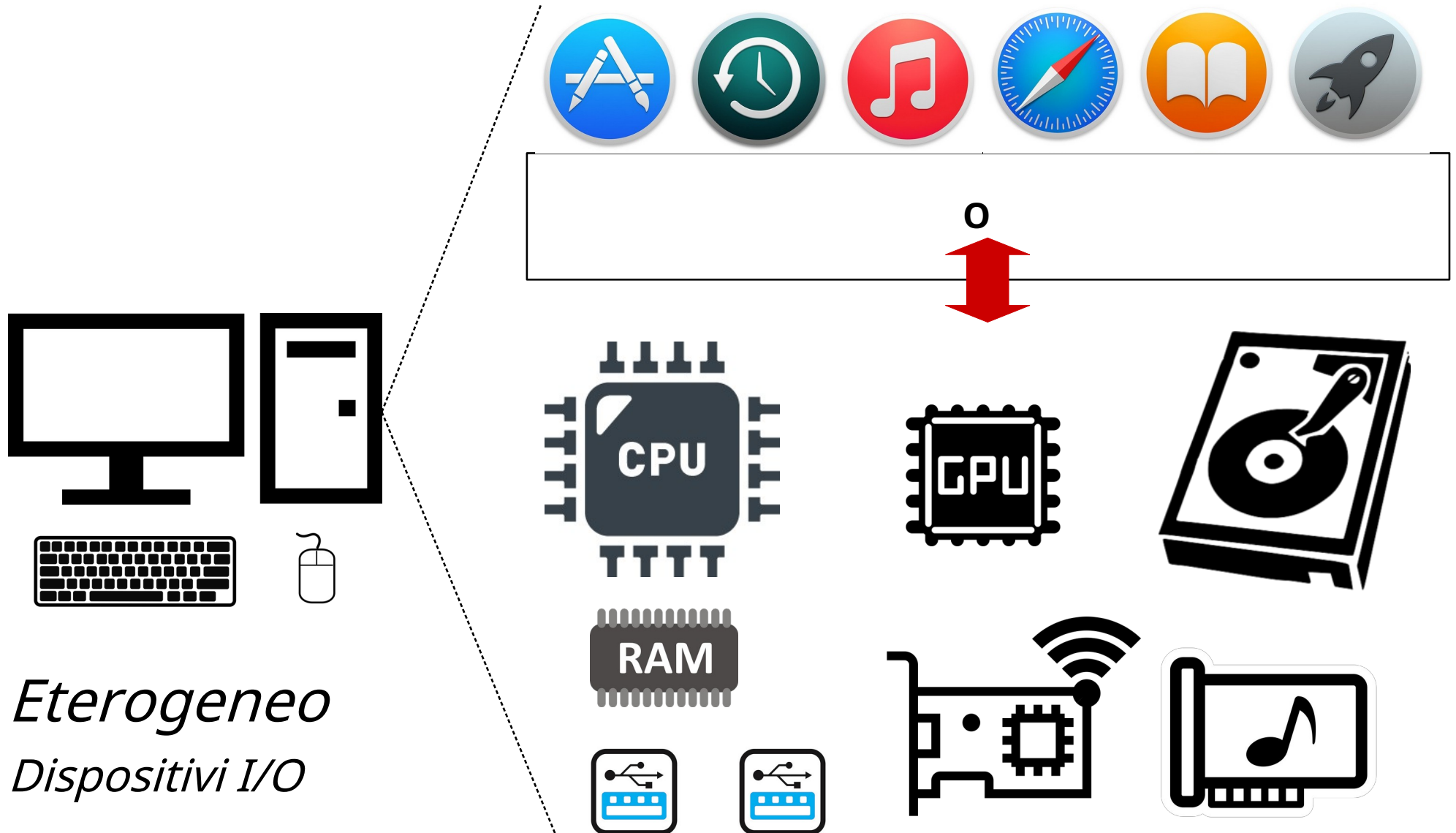
- **ESEMPIO:** Dispositivi di input e di output_(continua)
 - Per ciascun dispositivo, indicare se viene generalmente utilizzato per l'ingresso (I), l'uscita (O) o entrambi (B)
 - **[IO]** Tastiera
 - **[O]** Altoparlante
 - **[O]** Schermo
 - **[B]** Disco rigido
 - **[IO]** Microfono
 - **[B]** Scheda di interfaccia di rete (NIC)
 - **[B]** Schede di memoria (ovvero schede flash)

Anche "**B**" va bene se consideriamo i touchscreen

Sottosistema I/O (continua)

- Il sistema operativo deve gestire e controllare i dispositivi I/O e le operazioni I/O
 - Emettere comandi ai dispositivi I/O
 - Cattura le interruzioni
 - Gestire gli errori
- Un problema impegnativo
 - I dispositivi I/O variano ampiamente nella loro funzione e velocità
 - Sono necessari metodi diversi per controllarli

Sottosistema I/O



Sottosistema I/O (continua)

Velocità eterogenee

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Bluetooth 5 BLE	256 KB/sec
Scanner at 300 dpi	1 MB/sec
Digital video recorder	3.5 MB/sec
802.11n Wireless	37.5 MB/sec
USB 2.0	60 MB/sec
16x Blu-ray disc	72 MB/sec
Gigabit Ethernet	125 MB/sec
SATA 3 disk drive	600 MB/sec
USB 3.0	625 MB/sec
Single-lane PCIe 3.0 bus	985 MB/sec
802.11ax Wireless	1.25 GB/sec
PCIe Gen 3.0 NVMe M.2 SSD (reading)	3.5 GB/sec
USB 4.0	5 GB/sec
PCI Express 6.0	126 GB/sec

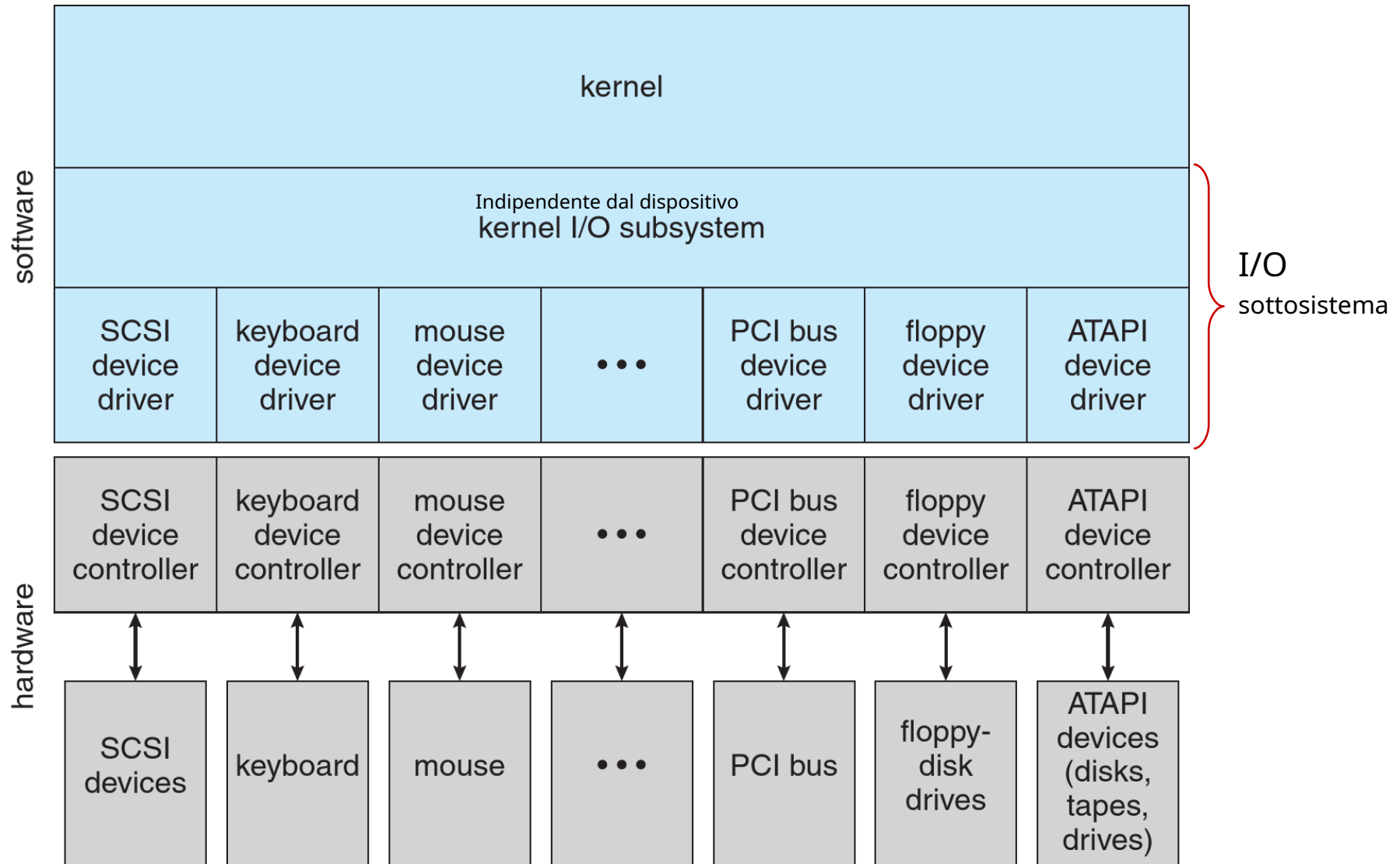
Fonte: [MOS5, Fig. 5-1]

Sottosistema I/O (continua)

- **Sottosistema I/O**

- Parte del sistema operativo che separa il resto del sistema operativo dalle complessità della gestione dei dispositivi I/O
- Deve fornire un'interfaccia uniforme ai livelli superiori per accedere a tutti i dispositivi I/O
- Deve essere in grado di comunicare con dispositivi I/O molto diversi
 - Dispone di gestori dedicati (**autisti**) ognuno di essi sovrintende ai protocolli necessari per gestire uno specifico dispositivo I/O

Sottosistema I/O (continua)



Sottosistema I/O (continua)

Una metafora visiva



Catena di montaggio

- *Protocolli*
 - Come/quali parti arrivano
 - Come/quali prodotti escono
- *Gestori dedicati*
 - Personale dedicato
- *Disaccoppiare i dettagli I/O dall'elaborazione principale*
 - Estrae i dettagli di spedizione dalla realizzazione dei prodotti

Sottosistema I/O

- *Protocolli*
 - Interfacce per I/O del dispositivo
- *Gestori dedicati*
 - I driver di periferica
- *Disaccoppiare i dettagli I/O dall'elaborazione principale*
 - Estrae i dettagli del dispositivo I/O dalle applicazioni

Sottosistema I/O (continua)

- **Blocca i dispositivi**

- Operare con granularità di dimensione fissa *blocchi* di dati
- Un blocco è la più piccola unità indirizzabile
- I trasferimenti sono espressi in unità di interi blocchi
- Un blocco può essere letto/scritto indipendentemente da tutti gli altri (→ accesso casuale)

- **Dispositivi di carattere (Dispositivi orientati al flusso)**

- Consegna o accetta flusso di byte (caratteri), indipendentemente da qualsiasi struttura a blocchi
- Non indirizzabile
- Le operazioni tipiche sono get/put carattere
- Non è presente alcuna operazione di ricerca (→ accesso seriale)

Sottosistema I/O (continua)

- **ESEMPIO:** Dispositivi a blocchi e a carattere
 - Per ogni dispositivo, indicare se si tratta di un dispositivo a blocchi (B), o di un dispositivo a caratteri (C). Utilizzo "?" Quando non sei sicuro
 - []Topo
 - []Disco rigido
 - []Tastiera
 - []Unità DVD
 - []Disco USB
 - []NIC

Sottosistema I/O (continua)

- **ESEMPIO:** Dispositivi a blocchi e a carattere (continua)
 - Per ogni dispositivo, indicare se si tratta di un dispositivo a blocchi (B), o di un dispositivo a caratteri (C). Utilizzo "?" Quando non sei sicuro
 - [C]Topo
 - [B]Disco rigido
 - [C]Tastiera
 - [B]Unità DVD
 - [B]Disco USB
 - [?]NIC

Sottosistema I/O (continua)

- **ESEMPIO:** Dispositivi a blocchi e a carattere_(continua)
 - *Blocca i dispositivi:* dischi rigidi, dischi DVD, chiavette USB, ...
 - Ad esempio, possiamo leggere qualsiasi blocco su un disco rigido indipendentemente dalla posizione attuale della testina di lettura
 - *Dispositivi di carattere:* tastiere, mouse, stampanti, ...
 - Ad esempio, una volta letto un byte dal mouse, non può essere letto nuovamente
 - Allo stesso modo, non possiamo leggere i precedenti N caratteri digitati su una tastiera
 - *Dispositivi di rete:* un caso speciale nel mezzo
 - Consegnano/ricevono più di un carattere alla volta come pacchetti, ma la loro granularità non è necessariamente un blocco di dimensione fissa e non supportano l'accesso casuale
 - È come un flusso di blocchi di dati di dimensioni potenzialmente diverse

Sottosistema I/O (continua)

- **ESEMPIO:** Dispositivi a blocchi e a carattere(continua)

- Blocca i dispositivi

```
FILE*fp =NULL; int  
ret = 0;  
...  
fp =fopen("/dev/sda", "rb"); /* Apre il dispositivo disco */  
ret = fseek(fp, 0L, SEEK_END); /* Cerca alla fine del file */ Se(ret == -1)  
perror("Impossibile cercare il dispositivo"); fchiudi(fp);
```

→ *OK!*

- Dispositivi di carattere

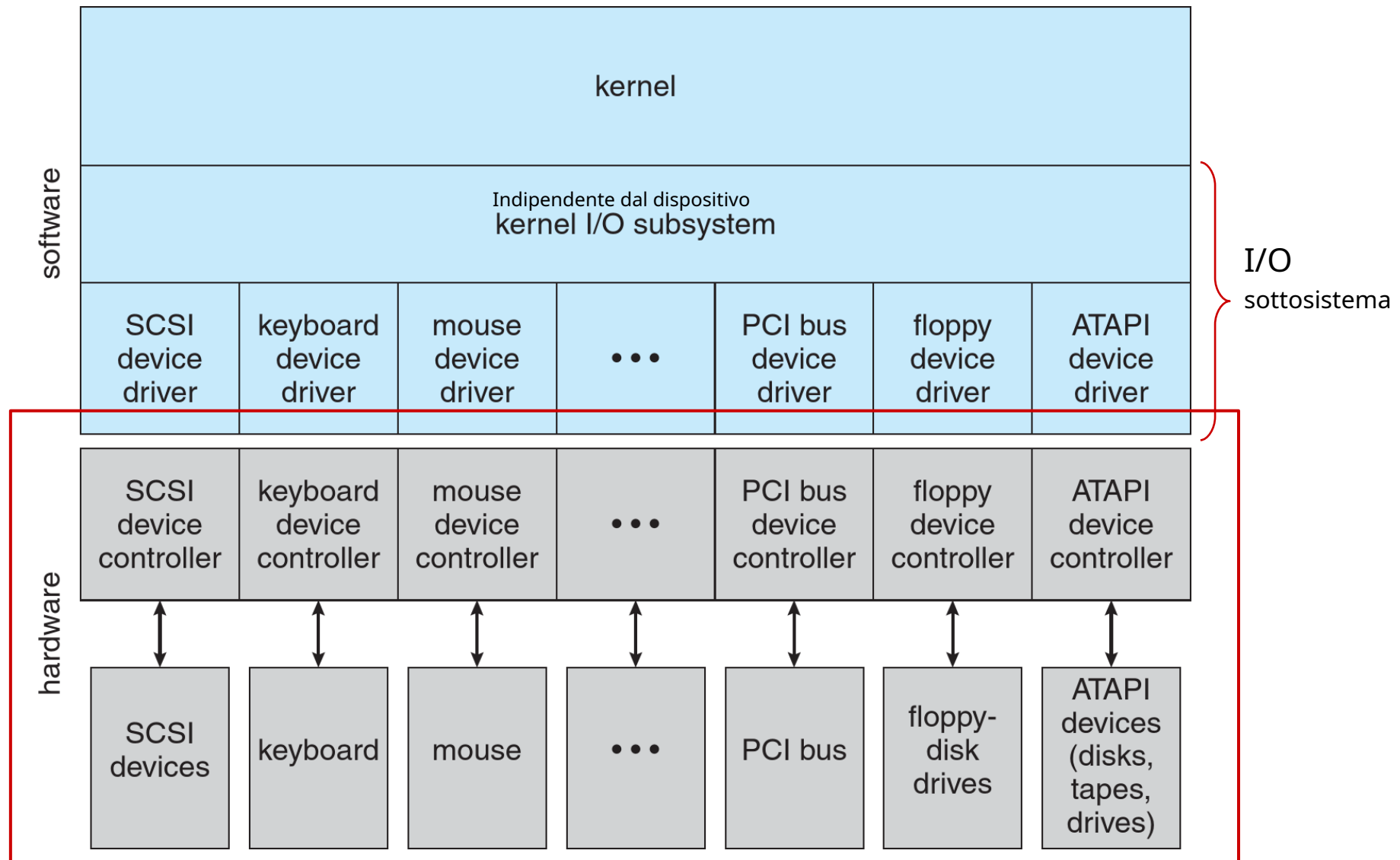
```
FILE*fp =NULL; int  
ret = 0;  
...  
fp =fopen("/dev/tty", "rb"); /* Apre il dispositivo terminale */ ret = fseek(fp, 0L,  
SEEK_END); /* Cerca alla fine del file */ Se(ret == -1) perror("Impossibile cercare il  
dispositivo"); fchiudi(fp);
```

→ *Impossibile cercare il dispositivo: ricerca illegale*

Sottosistema I/O (continua)

- Classificazione imperfetta
 - Alcuni dispositivi non si adattano
 - Dispositivi di rete, orologi,...
- Ma abbastanza generale da poter essere utilizzato come base per rendere indipendenti dal dispositivo quelle parti del sistema operativo che si occupano di I/O di alto livello
 - Ad esempio, il file system (ovvero la parte del sistema operativo che gestisce i file) si occupa solo di dispositivi a blocchi astratti e lascia la parte dipendente dal dispositivo al software di livello inferiore

Hardware di I/O (continua)



Hardware di I/O (continua)

- Un dispositivo I/O è generalmente costituito da due parti:
 - IL**dispositivo** stesso (componenti meccanici)
 - IL**controllore**(componenti elettronici)

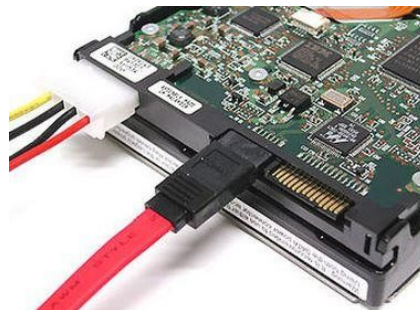
Hardware I/O: dispositivo

- Il **dispositivo** comunica con il computer attraverso il suo *controllore*, tramite un punto di connessione, denominato **porta**
 - Porta USB, porta parallela,...
- Di solito, il dispositivo è dotato di un connettore al quale è possibile collegare al computer un cavo che porta al dispositivo stesso

Connessione VGA



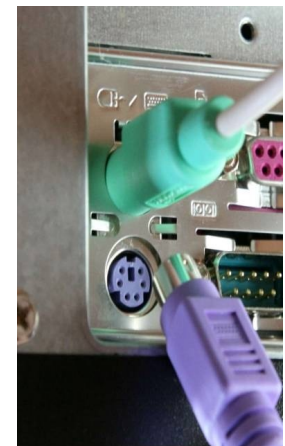
Connessione SATA



Connessione USB



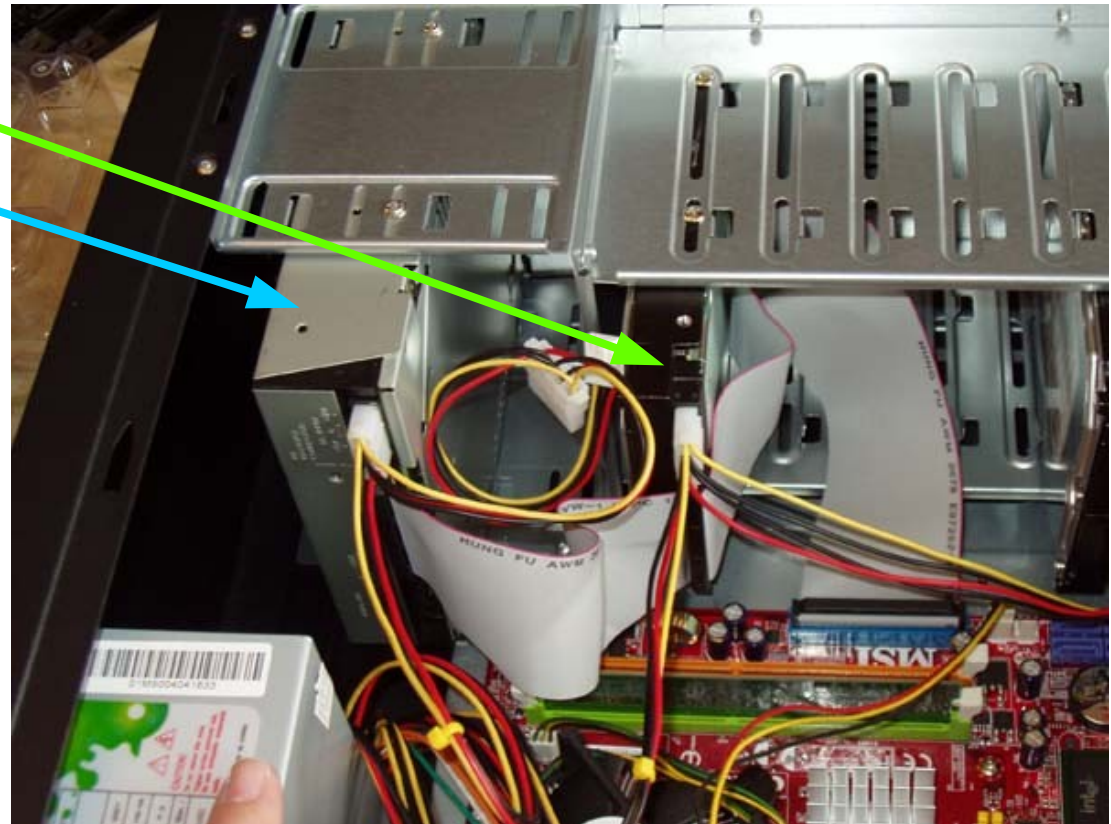
Connessione PS/2



Hardware I/O: dispositivo (continua)

- Se i dispositivi condividono un set comune di cavi, la connessione viene denominata **autobus**

Un'unità CD-ROM e un'unità disco rigido collegati tramite un cavo bus ATA (noto anche come IDE o PATA).



ACRONIMI

- **IDE:** Elettronica di azionamento integrata
- **ATA:** Avanzate Allegato tecnologico
- **PATA:** ATA parallelo

Hardware I/O: controller del dispositivo

- **ILcontrollore(Oadattatore)** è un chip o un insieme di chip che controlla fisicamente il dispositivo
 - Spesso contiene un piccolo computer incorporato programmato per controllare il dispositivo

Hardware I/O: controller dei dispositivi

(continua)

- Il controller accetta comandi dal sistema operativo e li esegue
 - L'interfaccia tra il controller e il dispositivo è solitamente standard, ma di solito è anche di livello molto basso
 - Il controller presenta un'interfaccia più semplice per il sistema operativo nascondendo gran parte dei dettagli e della complessità del dispositivo

Hardware I/O: controller dei dispositivi

(continua)

- **ESEMPIO:** Il sistema operativo desidera leggere un settore del disco rigido
 - Il sistema operativo vede il disco rigido come una sequenza lineare di settori, ciascuno dei quali ha una dimensione di 512B
 - L'indirizzamento dei settori inizia da 0 e aumenta linearmente
 - All'interno del disco rigido,
 - Ogni settore è indirizzato da una tripla:
<cilindro, testa, settore>
 - La struttura di ciascun settore è:
<preambolo> + <dati> + <checksum>
 - *<preambolo>*: contiene il cilindro, il numero del settore, ...
 - *<data>*: contiene dati (512B in questo caso)
 - *<checksum>*: utilizzato per il controllo degli errori

Hardware I/O: controller dei dispositivi

(continua)

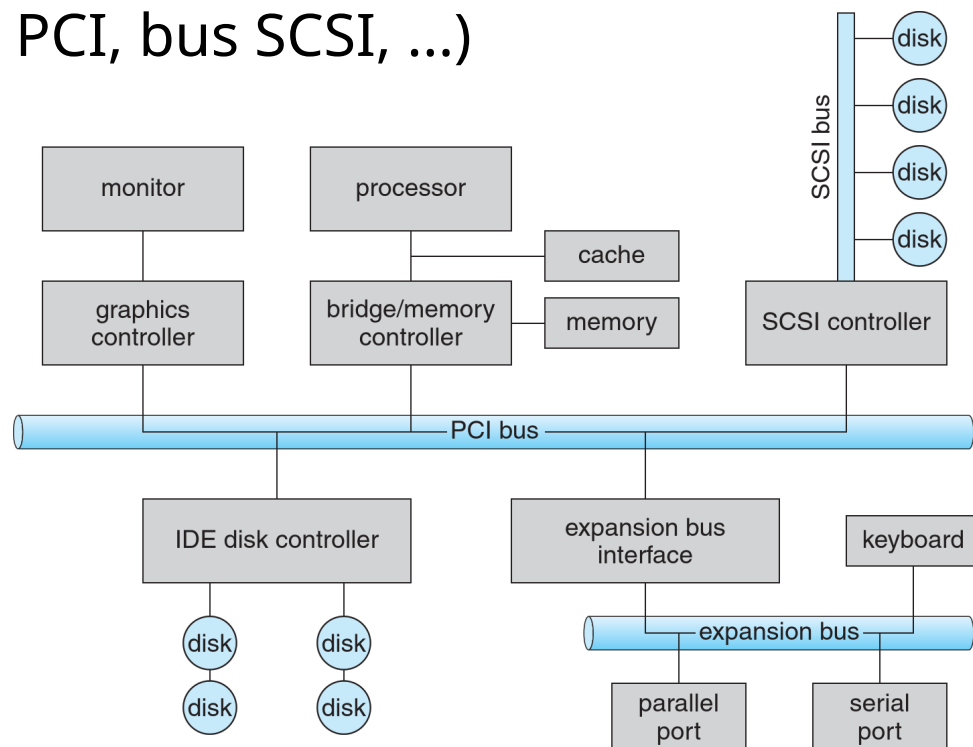
- **ESEMPIO:** Il sistema operativo desidera leggere un settore del disco_(continua)
 - Il sistema operativo chiede al controller del disco di leggere il settore 11206 dal disco
 - Controller del disco:
 1. Converti il settore lineare numero 11206 nella rappresentazione interna, ad esempio:
<cilindro x, testa y, settore z>
→ Di solito, $z \neq 11206$
 2. Rileva la posizione attuale della testa
 3. Spostare la testa dalla posizione attuale a quella destra
 4. Attendere la rotazione del settore apposito sotto la testa
 5. Leggere il flusso di bit associato al settore destro e memorizzarlo nel buffer interno
 6. Esegue il controllo degli errori
 7. Se OK restituisce <dati> del settore letto, altrimenti riprova

Hardware I/O: controller dei dispositivi

(continua)

- La CPU e i controller del dispositivo comunicano tra loro tramite il bus di sistema
- I moderni sistemi informatici in genere dispongono di più bus che possono definire diversi protocolli di comunicazione (ad esempio, bus PCI, bus SCSI, ...)

- **UNBus PCI (Peripheral Component Interconnect).** (il bus di sistema comune del PC) collega il sottosistema processore-memoria a dispositivi veloci
 - Ora sostituito da **PCI Express (PCIe)**
- **UNbus di espansione** collega dispositivi relativamente lenti, come la tastiera e le porte seriali e USB



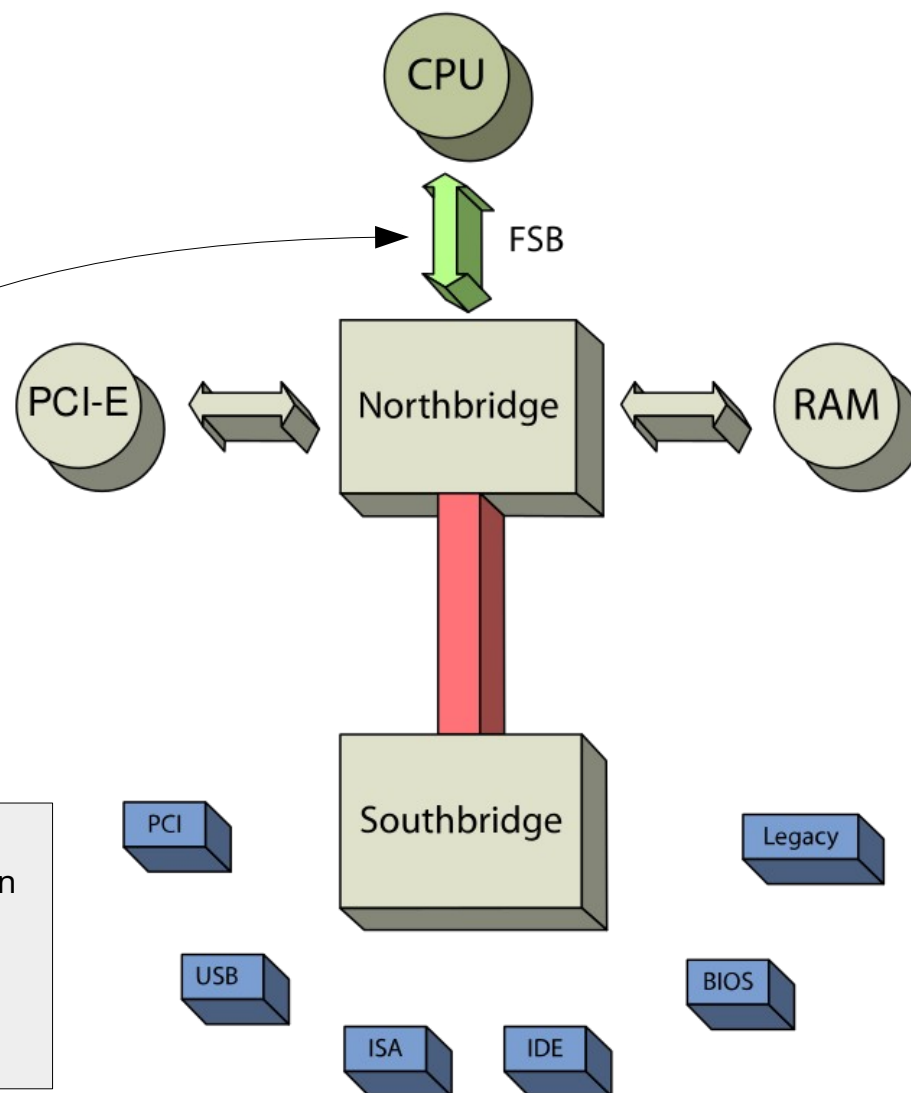
Hardware I/O: controller dei dispositivi

(continua)

Architettura del chipset di vecchia generazione

- **Ponte Nord**(ovvero**Hub del controller di memoria**): controller per comunicazioni ad alta velocità (ad esempio, CPU-RAM)
- **Ponte Sud**(ovvero**Hub del controller I/O**): controller per comunicazioni a bassa velocità (ad esempio, dispositivi CPU-I/O)
- **Autobus sul lato anteriore(FSB)**: connessione tra CPU e northbridge

Con l'aumento della velocità della CPU nel tempo, le comunicazioni tra CPU e northbridge sono diventate un problema *collo di bottiglia* a causa delle limitazioni imposte dalla trasmissione dei dati tra la CPU e il suo chipset (ad esempio, comunicazioni CPU-RAM limitate dalla velocità dell'FSB)



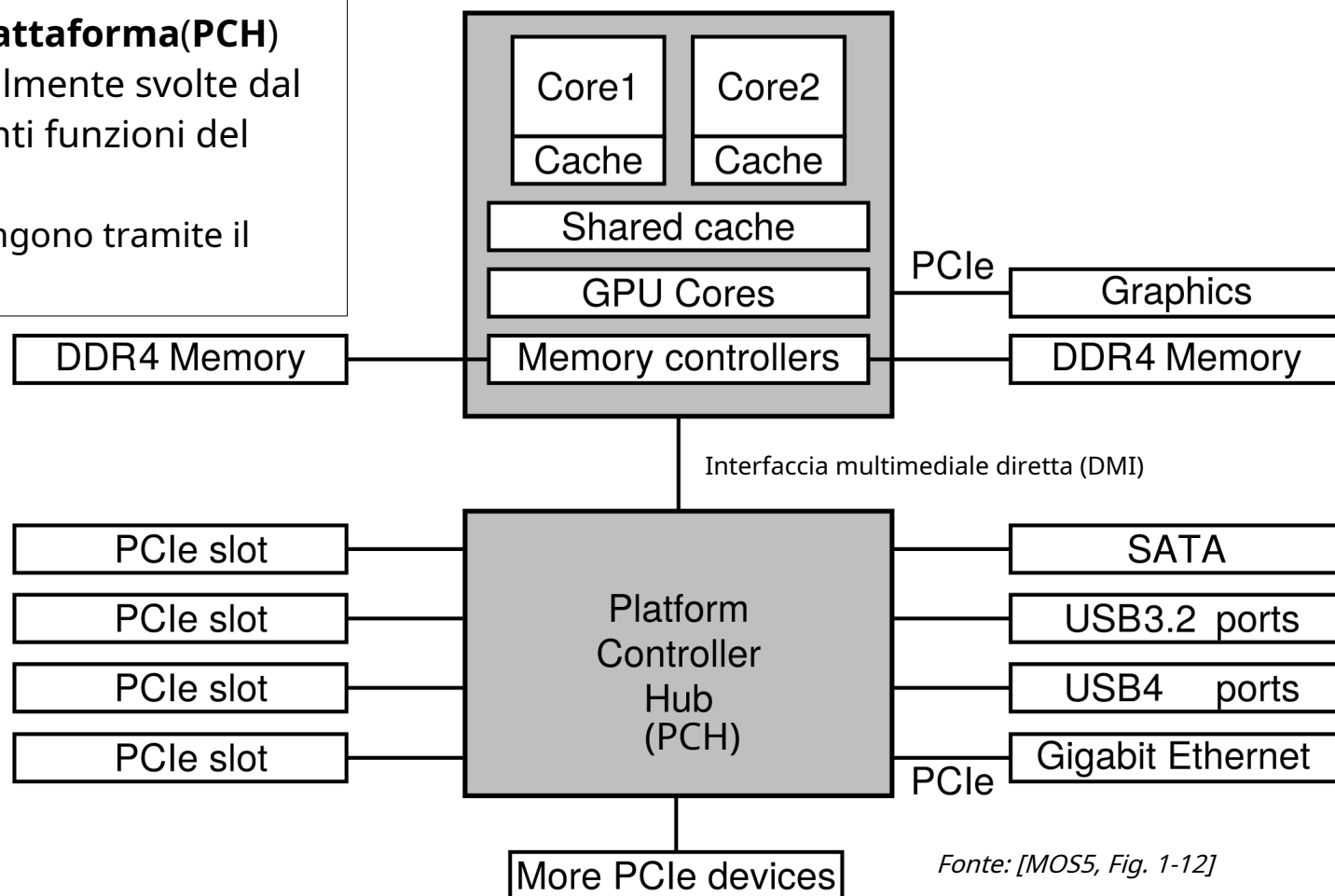
Fonte: <https://commons.wikimedia.org/w/index.php?curid=35241077>

Hardware I/O: controller dei dispositivi

(continua)

Architetture di chipset moderne

- La maggior parte delle funzioni del Northbridge (ad esempio, controller di memoria e linee PCIe per schede di espansione) sono integrate nella CPU
- **IL Hub di controllo della piattaforma (PCH)** svolge le funzioni tradizionalmente svolte dal Southbridge e dalle rimanenti funzioni del Northbridge
- I trasferimenti CPU-PCH avvengono tramite il bus DMI



Fonte: [MO55, Fig. 1-12]

Hardware I/O: controller dei dispositivi

(continua)

- Un controllore può assumere diverse forme
 1. Un singolo chip (o porzione di chip) nella scheda madre del computer che controlla i segnali sui fili di una porta a cui è collegato il dispositivo
 - Ad esempio, controller USB e FireWire (IEEE 1394).
 - Parte del chipset della scheda madre
 - Sulle vecchie architetture: **Ponte Sud**
 - Sulle architetture moderne: **Hub di controllo della piattaforma**
 - Sulle architetture di prossima generazione: integrato nel pacchetto CPU
 - Ad esempio, a partire da Broadwell (2014) e Skylake (2015) di Intel

Hardware I/O: controller dei dispositivi

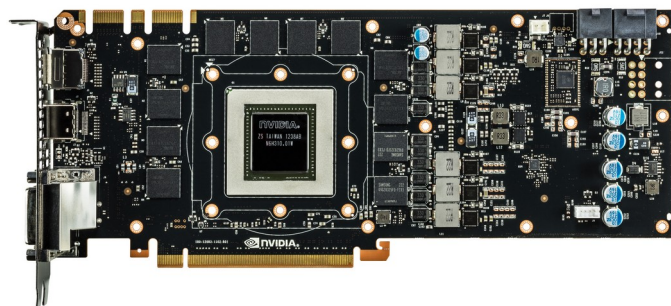
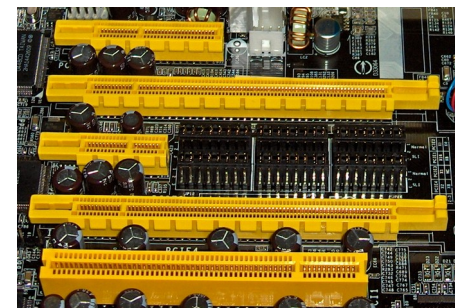
(continua)

- Un controllore può assumere diverse forme(continua)
 2. Un circuito stampato separato che si collega al computer tramite uno slot di estensione
 - Di solito chiamato **adattatore bus host** (o **adattatore ospite**)
 - Necessario quando il protocollo utilizzato per comunicare con il dispositivo è complesso
 - Ad esempio, adattatore SCSI, adattatore PCIe
 - Solitamente contiene un processore e una memoria privata per consentirgli di elaborare i messaggi del protocollo

Fonte: Di w:user:snickerdo (<https://commons.wikimedia.org/w/index.php?curid=923562>)

Il circuito stampato
dell'adattatore grafico
NVIDIA GTX 780

Vari slot PCI Express (PCIe) sulla
scheda madre del computer



Fonte: di GBPublic_PR (<https://commons.wikimedia.org/w/index.php?curid=31762739>)

Hardware I/O: controller dei dispositivi

(continua)

- Alcuni dispositivi dispongono di controller integrati (**controllori periferici**)
 - Consente al controller sul computer (**controllore ospite**) per inviare una serie di comandi di livello superiore al controller della periferica
 - Il controller periferico mappa questi comandi di alto livello su quelli di basso livello
 - Ad esempio, nelle unità disco, viene utilizzato per eseguire il controllo degli errori, la memorizzazione nella cache e il buffering

Controller integrato del disco Serial ATA (SATA).



Hardware I/O: Interazione CPU – Dispositivi

Come funzionano CPU e dispositivi I/O
comunicare?

Hardware I/O: Interazione CPU – Dispositivi

(continua)

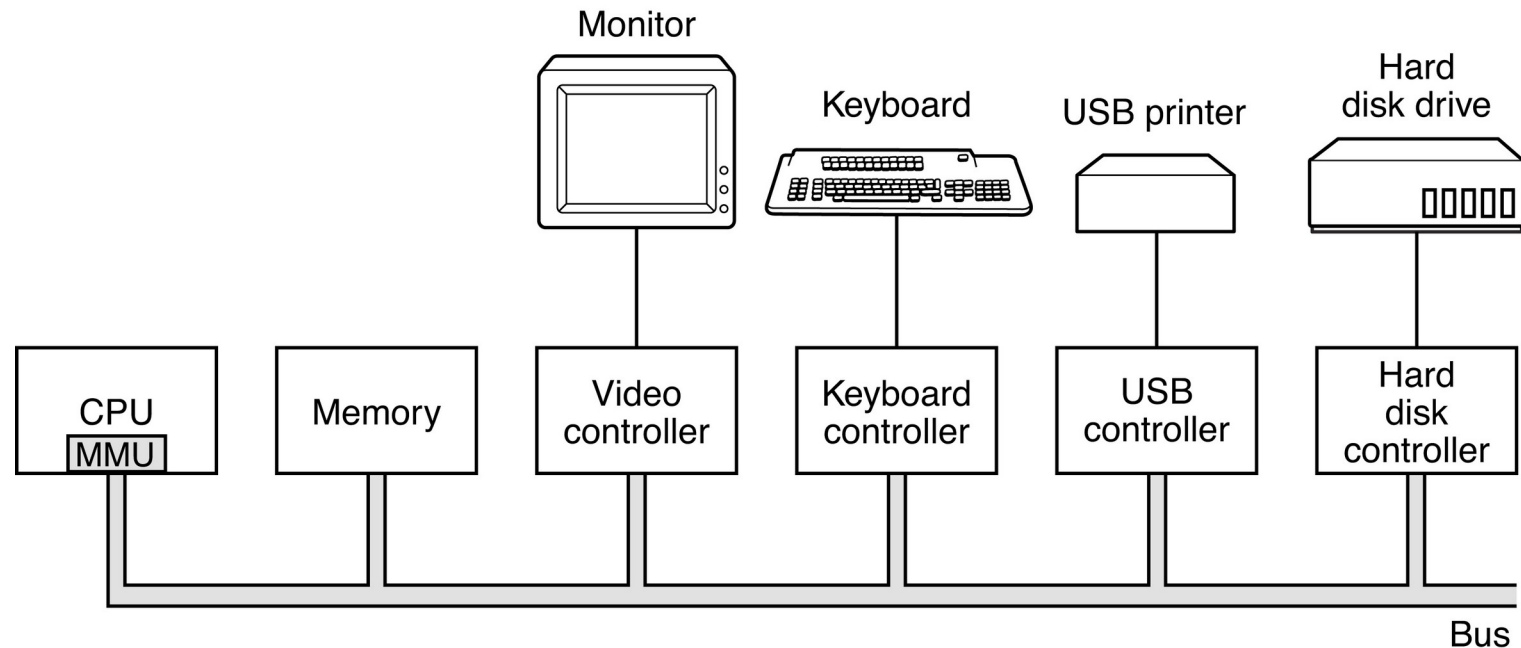
- **Modelli di riferimento**

- I dispositivi sono estremamente eterogenei
 - Differenti architetture HW
 - Diverse interfacce per interagire con loro
 - Velocità diverse
 - Diversi tipi di funzionalità che forniscono
- Per semplificare la nostra discussione, usiamo **modelli di riferimento** per il *Architettura di sistema per dispositivi*
 - Questi modelli ci consentono di evidenziare le caratteristiche chiave che consentono l'integrazione dei dispositivi in un sistema
 - Salvo diversa indicazione, faremo sempre riferimento a questi modelli nel resto di questo corso

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Modello di riferimento del sistema**
 - CPU, memoria e dispositivi I/O sono tutti collegati da a **bus di sistema unico** (ad esempio, dal bus PCIe) e comunicano tra loro tramite esso

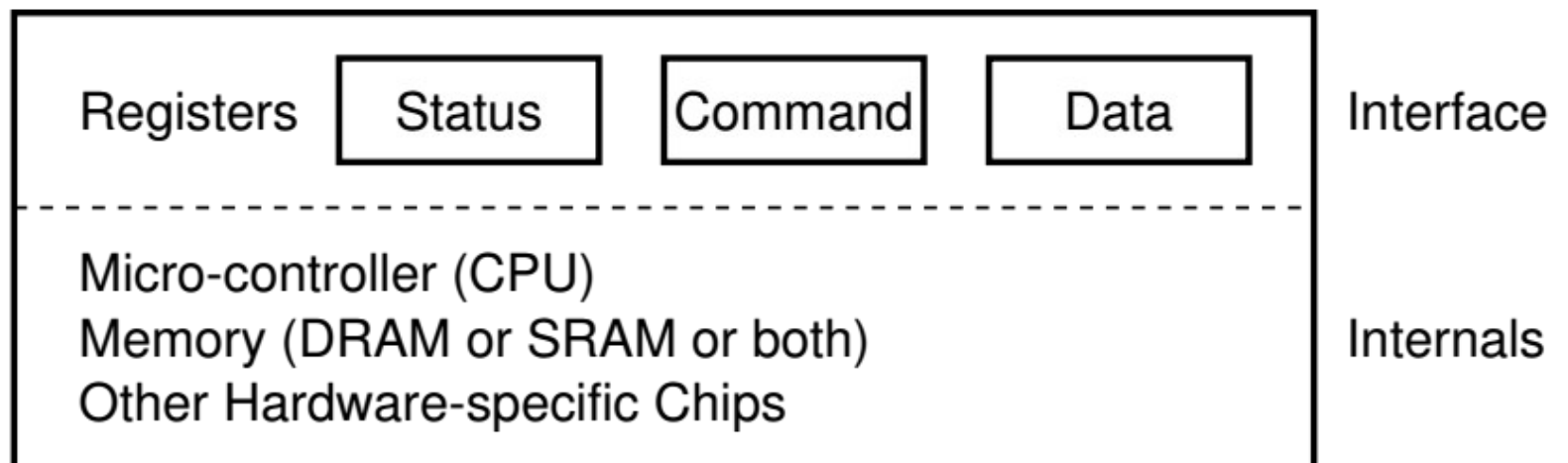


Fonte: [MOS5, Fig. 1-6]

Hardware I/O: Interazione CPU – Dispositivi

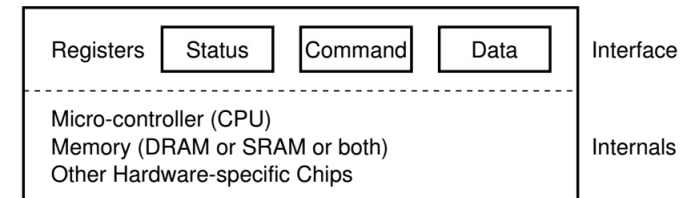
(continua)

- **Modello di riferimento del dispositivo**
 - Il **interfaccia** è l'interfaccia HW che il controller del dispositivo presenta al resto del sistema e che può essere utilizzata per controllare il funzionamento del dispositivo
 - Il **struttura interna** è specifico dell'implementazione ed è responsabile dell'implementazione dell'astrazione che il controller del dispositivo presenta al sistema



Hardware I/O: Interazione CPU – Dispositivi

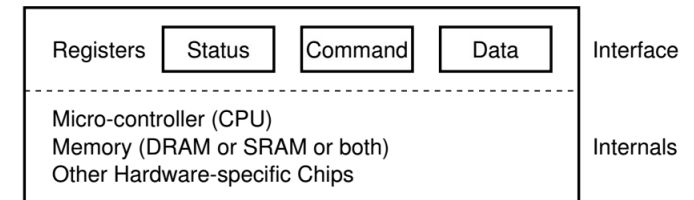
(continua)



- **Modello di riferimento del dispositivo**_(continua)
 - Il controller del dispositivo ne ha alcuni **registri di controllo** a cui può accedere la CPU e che consente l'interazione CPU-dispositivo:
 - I **registri di dati** sono utilizzati dalla CPU per controllare il trasferimento dei dati in entrata e in uscita dal dispositivo
 - Vengono letti/scritti dalla CPU per ottenere input/inviare output
 - I **registri di stato** vengono utilizzati dalla CPU per scoprire cosa sta succedendo sul dispositivo
 - Contengono bit che la CPU può leggere per vedere lo stato attuale del dispositivo (il comando è stato completato, si è verificato un errore, pronto ad accettare un nuovo comando, ...)
 - I **registri di comando** può essere scritto dalla CPU per controllare cosa farà esattamente il dispositivo
 - Ad esempio, per avviare un comando o modificare la modalità di un dispositivo (impostare la comunicazione full-duplex o half-duplex, impostare l'operazione di lettura/scrittura, ...)

Hardware I/O: Interazione CPU – Dispositivi

(continua)



- **Modello di riferimento del dispositivo**_(continua)
 - Oltre a questi registri, molti dispositivi hanno un **file buffer dei dati**
 - Utilizzato per archiviare i dati letti dal dispositivo che non possono essere passati direttamente al sistema operativo
 - Ad esempio, per archiviare il flusso di byte in entrata da una connessione di rete per assemblarli in un pacchetto di rete che il sistema operativo possa comprendere
 - Utilizzato per memorizzare i dati da scrivere sul dispositivo
 - Ad esempio, per scrivere dati a velocità costante su dispositivi audio digitali
 - Ad esempio, per scrivere pixel sullo schermo video

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- Per abilitare l'interazione CPU-dispositivo, i registri del dispositivo devono essere accessibili dalla CPU
- Due approcci principali:
 - I/O mappato sulle porte
 - I/O mappato in memoria
- Può essere combinato (**approccio ibrido**)

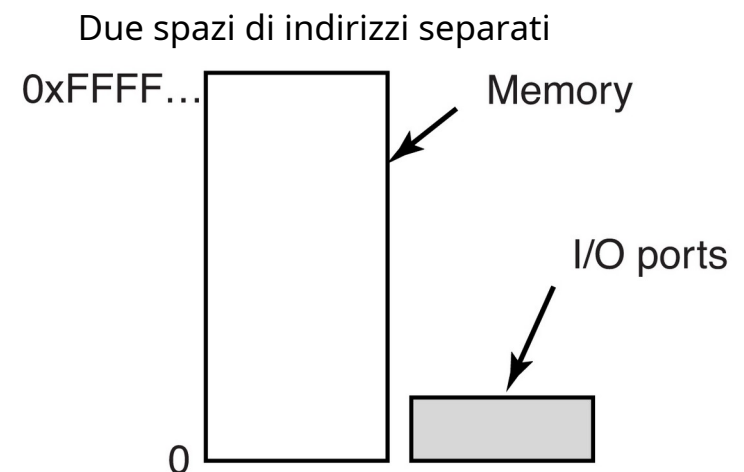
Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O mappato sulle porte**

- Ad ogni registro del dispositivo viene assegnato un **Numero della porta I/O**

- Il numero di porta I/O è un numero intero (ad esempio, un numero intero a 8 o 16 bit) generalmente predefinito o assegnato al momento dell'avvio
- L'insieme di tutti i numeri delle porte I/O forma il **I/O spazio portuale**
- Accessibile solo in modalità kernel
- Spazio porta I/O $\neq$ Spazio di memoria



(a)

Fonte: [MOS5, Fig. 5-2]

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O mappato sulle porte**_(continua)

- Porte I/O tipiche (elenco incompleto)

I/O address range (hexadecimal)	device
000–00F	DMA controller
020–021	interrupt controller
040–043	timer
200–20F	game controller
2F8–2FF	serial port (secondary)
320–32F	hard-disk controller
378–37F	parallel port
3D0–3DF	graphics controller
3F0–3F7	diskette-drive controller
3F8–3FF	serial port (primary)

Fonte: [OSC9, Fig. 13.2]

- Consulta le specifiche del chipset per lo spazio reale delle porte I/O
 - Ad esempio, vedere Sez. 9.3 della scheda tecnica del chipset Intel© serie 7
<http://www.intel.com/content/www/us/en/chipsets/c600-series-chipset-datasheet.html>

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O mappato sulle porte**(continua)
 - **Istruzioni I/O speciali** comunicare
 - Seleziona il dispositivo adatto
 - Sposta i bit dentro o fuori un registro del dispositivo
 - ***INREG, PORTO***
 - La CPU legge il registro del dispositivo *PORTA* e memorizzare il risultato nel registro della CPU *REG*
 - ***FUORI PORTO, REG***
 - La CPU scrive il contenuto del registro della CPU *REG* al registro del dispositivo *PORTA*
 - Questi sono **istruzioni privilegiate** (può essere eseguito solo in modalità kernel)

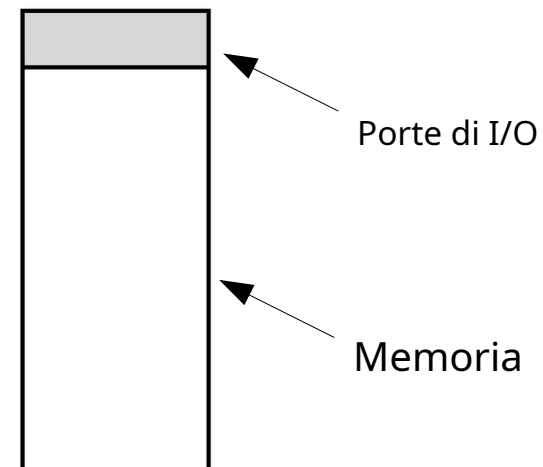
Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O mappato in memoria**

- I registri del dispositivo sono mappati nello spazio di memoria
- A ciascun registro viene assegnato un indirizzo di memoria univoco a cui non è assegnata alcuna memoria

- Solitamente eseguito al momento dell'avvio

One address space



(b) Fonte: [MOS5, Fig. 5-2]

Hardware I/O: Interazione CPU – Dispositivi

(continua)

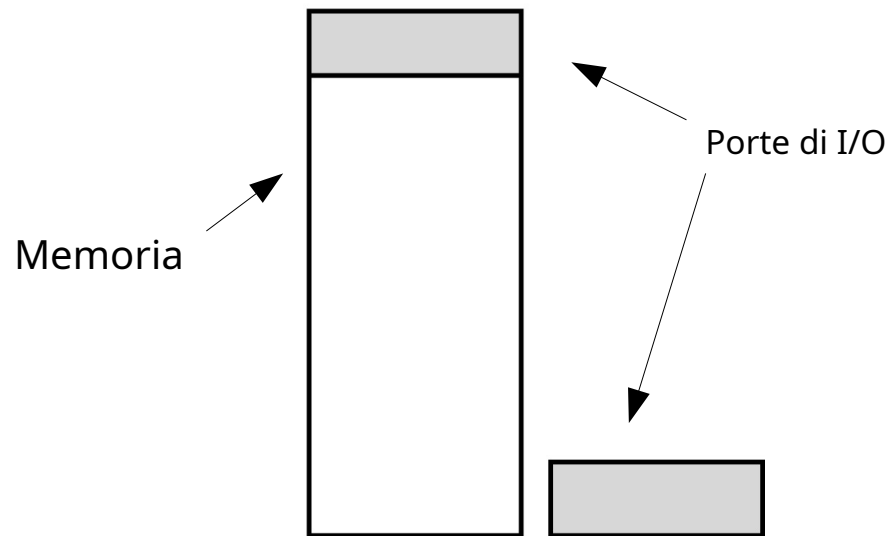
- **I/O mappato in memoria**_(continua)
 - **Istruzioni standard per il trasferimento dei dati**(cioè quelli utilizzati per gli accessi alla memoria) per leggere e scrivere i registri del dispositivo
 - ***MOVREG, INDIRIZZO***
 - La CPU legge il registro del dispositivo mappato sull'indirizzo di memoria *INDIRIZZO* e memorizzare il risultato nel registro della CPU *REG*
 - ***MOVINDIRIZZO, REG***
 - La CPU scrive il contenuto del registro della CPU *REG* al registro del dispositivo mappato all'indirizzo di memoria *INDIRIZZO*

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Approccio ibrido**

- Combina I/O mappati su porte e mappati in memoria
- Utilizza I/O con mappatura delle porte per alcuni dispositivi
- Utilizza I/O mappati in memoria per altri dispositivi

Two address spaces



(c)

UPO - OS2: I/O (parte 1) fonte: [MOS5, Fig. 5-2]

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Approccio ibrido**(continua)

- Buffer di dati mappati in memoria e porte I/O separate per i registri di controllo
- Utilizzato dall'architettura x86
- Ad esempio, controller grafico
 - I/O mappati sulle porte per operazioni di controllo di base
 - I/O mappati in memoria per contenere il contenuto dello schermo
 - Scrivere milioni di byte nella memoria grafica è più veloce che emettere milioni di istruzioni I/O

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO:** La CPU vuole leggere una parola situata nell'indirizzo X (X può essere un indirizzo di memoria o una porta I/O)
 1. La CPU inserisce l'indirizzo X sulle linee degli indirizzi del bus
 2. La CPU afferma un segnale READ sulla linea di controllo del bus
 3. Due casi:
 - Se I/O mappato sulle porte (promemoria: istruzioni e spazi di indirizzi diversi)
 - La CPU invia un altro segnale sulla linea di controllo di un altro bus per stabilire se è necessario spazio I/O o spazio di memoria
 - Se è spazio di memoria, la memoria risponde alla richiesta
 - Se si tratta di spazio I/O, il dispositivo I/O risponde alla richiesta
 - Se I/O mappato in memoria (promemoria: stesso spazio di indirizzi)
 - Ogni modulo di memoria e ogni dispositivo I/O confronta le linee di indirizzo con l'intervallo di indirizzi che serve
 - Se l'indirizzo rientra nel suo intervallo, risponde alla richiesta
 - Nessun conflitto possibile
 - Nessun indirizzo è assegnato sia alla memoria che al dispositivo I/O

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Punti di forza I/O mappati in memoria**
 - *Sviluppo semplificato dei driver di dispositivo*
 - *I driver di periferica*(li vedremo tra poco) possono essere implementati in linguaggi di programmazione di alto livello, tipicamente in C
 - I registri sono solo indirizzi di memoria e quindi possono essere visti come variabili nei linguaggi di programmazione di alto livello
- Con l'I/O mappato sulle porte, è necessario del codice assembly poiché lo spazio delle porte I/O $\neq$ lo spazio di memoria
 - Incline a errori
 - Nessun modo standard per inserire il codice assembly:
 - GCC GNU: `__asm__("...")`
 - MSVC++: `__asm {...}`
 - Meno portabile (il codice assembly dipende dall'architettura)

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Punti di forza I/O mappati in memoria**
 - *Non è necessario alcun meccanismo di protezione speciale per impedire ai processi utente di eseguire I/O di basso livello*
 - Per evitare che gli utenti affrontino il registro del dispositivo, il sistema operativo mappa semplicemente lo spazio degli indirizzi contenente i registri del dispositivo all'esterno dello spazio degli indirizzi virtuali di qualsiasi utente
 - Con l'I/O mappato sulle porte, sono necessari meccanismi speciali poiché lo spazio delle porte di I/O e lo spazio di memoria sono spazi di indirizzi diversi
 - Ad esempio, le istruzioni I/O sono privilegiate

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Punti di forza I/O mappati in memoria**
 - *Design della CPU semplificato ed economico*
 - Ogni istruzione che può fare riferimento alla memoria può anche fare riferimento ai registri del dispositivo
 - Inoltre, tutte le modalità di indirizzamento della CPU utilizzate per la memoria sono disponibili anche per gli I/O
 - Con l'I/O mappato sulle porte, nel set di istruzioni devono essere incluse istruzioni I/O speciali

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Punti di forza I/O mappati in memoria**

- *Design della CPU semplificato ed economico*_(continua)

- Ad esempio, per verificare se un dispositivo è inattivo dobbiamo testare a zero il registro mappato all'indirizzo X

- Con I/O mappato in memoria

CICLO: PROVA X	/* Controlla se il dispositivo è inattivo testando if addr. X è 0 */ /* Se è 0, vai su
BEQ PRONTO	PRONTO */
BRANCH LOOP	/* Altrimenti, continua il test */ PRONTO:...
	/* Invia al dispositivo inattivo */

- Con I/O mappato sulle porte

LOOP: IN REG, X	/* Legge il contenuto della porta X nel registro REG della CPU */ /*
PROVA REG	Controlla se il dispositivo è inattivo testando se REG è 0 */ /* Se è 0, vai a
BEQ PRONTO	READY */
BRANCH LOOP	/* Altrimenti, continua il test */ PRONTO:...
	/* Invia al dispositivo inattivo */

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Intensità I/O mappate sulle porte**
 - *Nessun intervallo di memoria consumato per I/O*
 - Non è necessario che i dispositivi consumino intervalli di indirizzi di memoria fisica
 - Utile in architetture con indirizzi di memoria fisica vincolati
 - Con l'I/O mappato in memoria, una parte della memoria fisica è riservata ai registri e ai buffer del dispositivo

Hardware I/O: Interazione CPU – Dispositivi

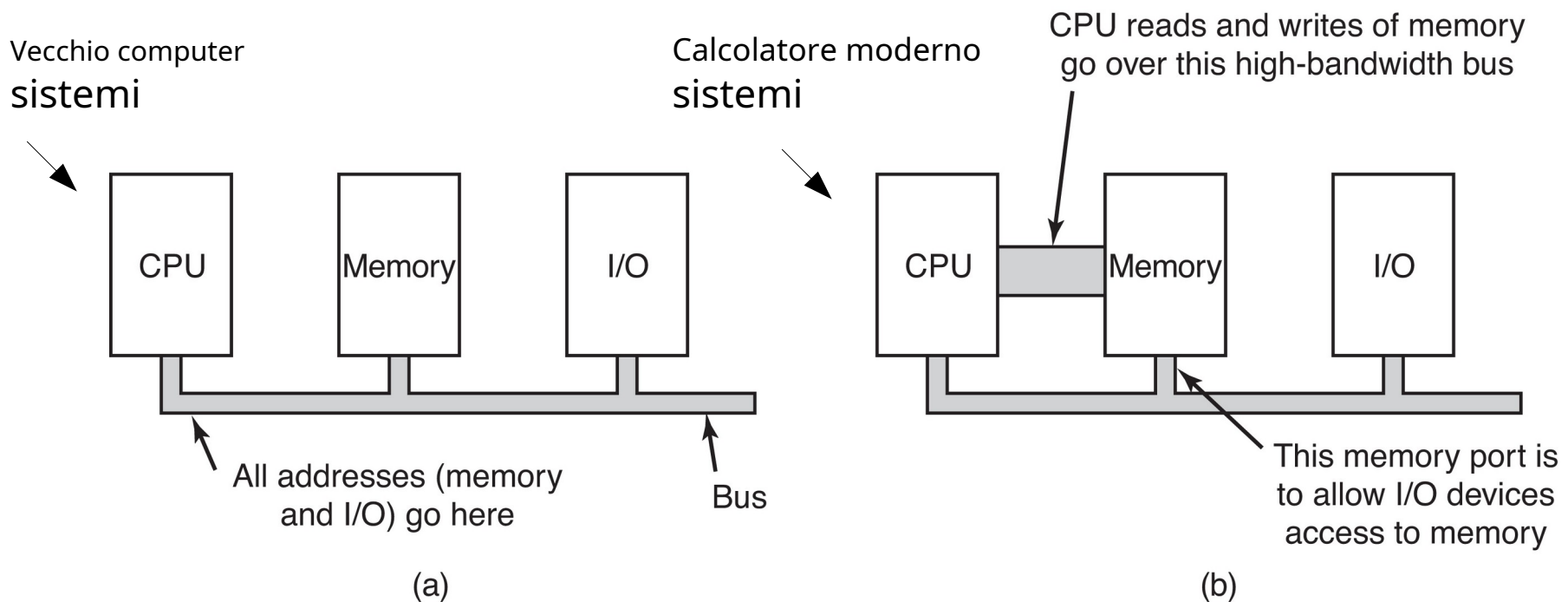
(continua)

- **Intensità I/O mappate sulle porte**
 - *Non è necessario gestire la memorizzazione nella cache*
 - Memorizzare nella cache un registro del dispositivo sarebbe disastroso
 - Ad esempio, un dispositivo potrebbe sembrare occupato anche se non è perché il relativo registro è stato memorizzato nella cache
 - Con l'I/O mappato sulle porte, lo spazio delle porte I/O non viene memorizzato nella cache
 - Con l'I/O mappato in memoria, l'hardware deve essere in grado di disabilitare selettivamente la memorizzazione nella cache
 - La gestione della memorizzazione nella cache selettiva aggiunge ulteriore complessità sia all'hardware che al sistema operativo

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- Intensità I/O mappate sulle porte
 - *Nessuna misura particolare in caso di autobus multipli*
 - I moderni sistemi informatici dispongono di un bus di memoria dedicato ad alta velocità per i trasferimenti di memoria CPU e di un altro bus lento per i trasferimenti CPU-I/O



Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Intensità I/O mappate sulle porte**
 - *Nessuna misura particolare in caso di autobus multipli*_(continua)
 - Con I/O mappato sulle porte, nessun problema
 - Lo spazio della porta I/O $\neq$ spazio di memoria
 - Le operazioni di I/O utilizzeranno il bus specifico
 - Con l'I/O mappato in memoria, a causa di un singolo spazio di indirizzi, i dispositivi I/O e i moduli di memoria devono esaminare i riferimenti di memoria per vedere a quali rispondere

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Intensità I/O mappate sulle porte**
 - *Nessuna misura particolare in caso di autobus multipli*(continua)
 - Varie opzioni da utilizzare con I/O mappati in memoria
 - Invia tutti i riferimenti alla memoria al bus di memoria e poi agli altri bus, se la memoria non risponde
 - Richiedono ulteriore complessità HW
 - Metti un dispositivo di snooping sul bus di memoria per passare tutti gli indirizzi presentati ai dispositivi I/O potenzialmente interessati
 - I dispositivi I/O potrebbero non essere in grado di elaborare le richieste alla velocità consentita dalla memoria
 - Filtra gli indirizzi "non di memoria" nel controller di memoria e inoltrali ai dispositivi I/O
 - È necessario determinare in fase di avvio quali indirizzi di memoria non sono realmente indirizzi di memoria ma sono invece riservati all'I/O

Hardware I/O: Interazione CPU – Dispositivi

(continua)

Come viene eseguito l'I/O?

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- Tre approcci
 - I/O programmati(ovvero sondaggio)
 - I/O basato su interrupt
 - I/O tramite DMA

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O programmati (PIO)**

- La CPU chiede al controller del dispositivo di eseguire una richiesta I/O e si trova in un ciclo stretto interrogando continuamente il dispositivo per vedere se è stata eseguita
 - Di solito c'è qualcosa nel file *registro di stato* ciò indica che il dispositivo è ancora occupato o che alcuni dati sono disponibili
- Una volta completata la richiesta di I/O, la CPU può eseguire l'istruzione successiva
 - Ad esempio, per inserire i dati (se presenti) leggerli dal dispositivo dove è necessario e restituirli al chiamante
- Questo metodo è anche chiamato **occupato in attesa** o **sondaggio**

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O programmati (PIO)**(continua)
 - Punti di forza
 - Facile da implementare
 - Buono per piccoli trasferimenti di dati
 - È perfettamente appropriato che nessuna azione sia possibile fino all'accesso I/O
 - Ad esempio, sistemi monouso
 - Punti deboli
 - La CPU non fa altro che controllare lo stato del dispositivo I/O finché non è pronto
 - La CPU non può essere utilizzata per svolgere attività più utili (ad esempio, pianificare un processo)

Hardware I/O: Interazione CPU – Dispositivi

(continua)

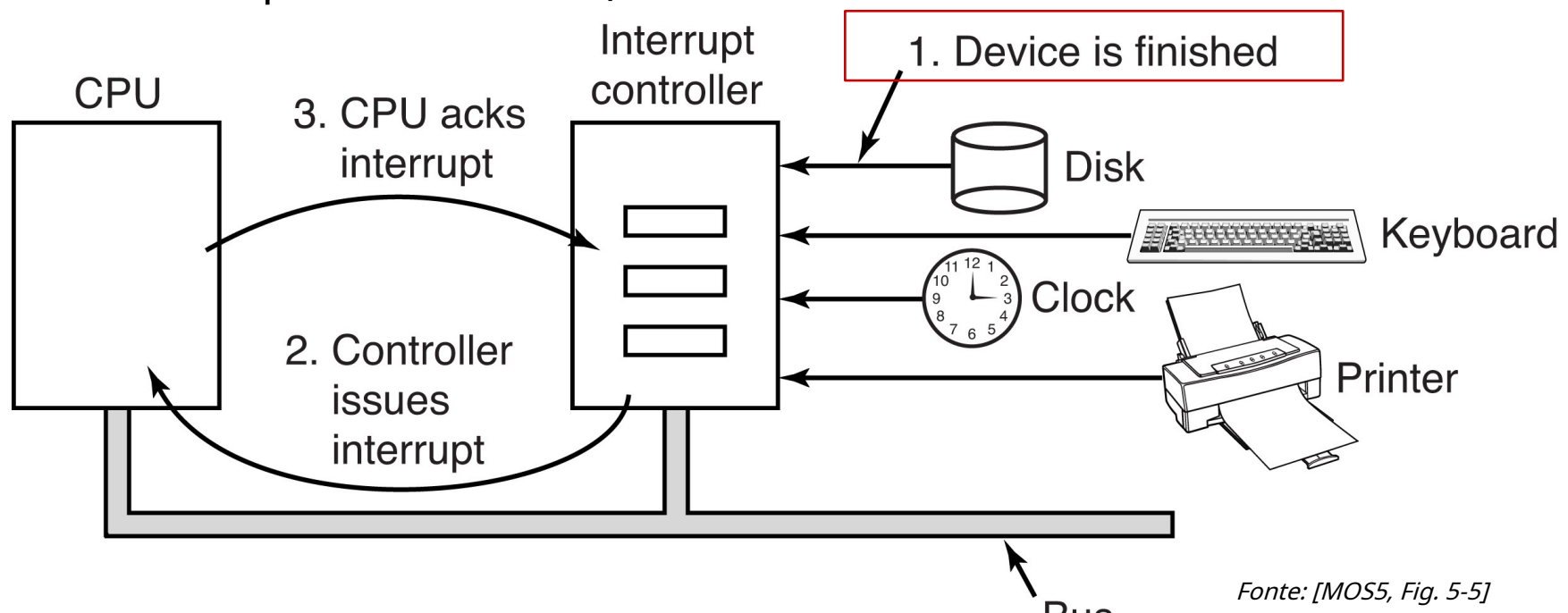
- **I/O basato su interrupt**

- La CPU chiede al controller del dispositivo di eseguire una richiesta di I/O, blocca il processo che ha richiesto l'operazione di I/O e quindi cerca altro lavoro da svolgere
 - Ad esempio, può pianificare un altro processo
- Quando il controller del dispositivo rileva la fine della richiesta I/O, genera un **interrompere** per segnalare il completamento

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O basato su interrupt**(continua)

- Per generare un'interruzione, il controller del dispositivo asserisce un segnale sulla linea di controllo del bus
- Il segnale viene rilevato da **controllore dell'interruzione**(parte del chipset del sistema)



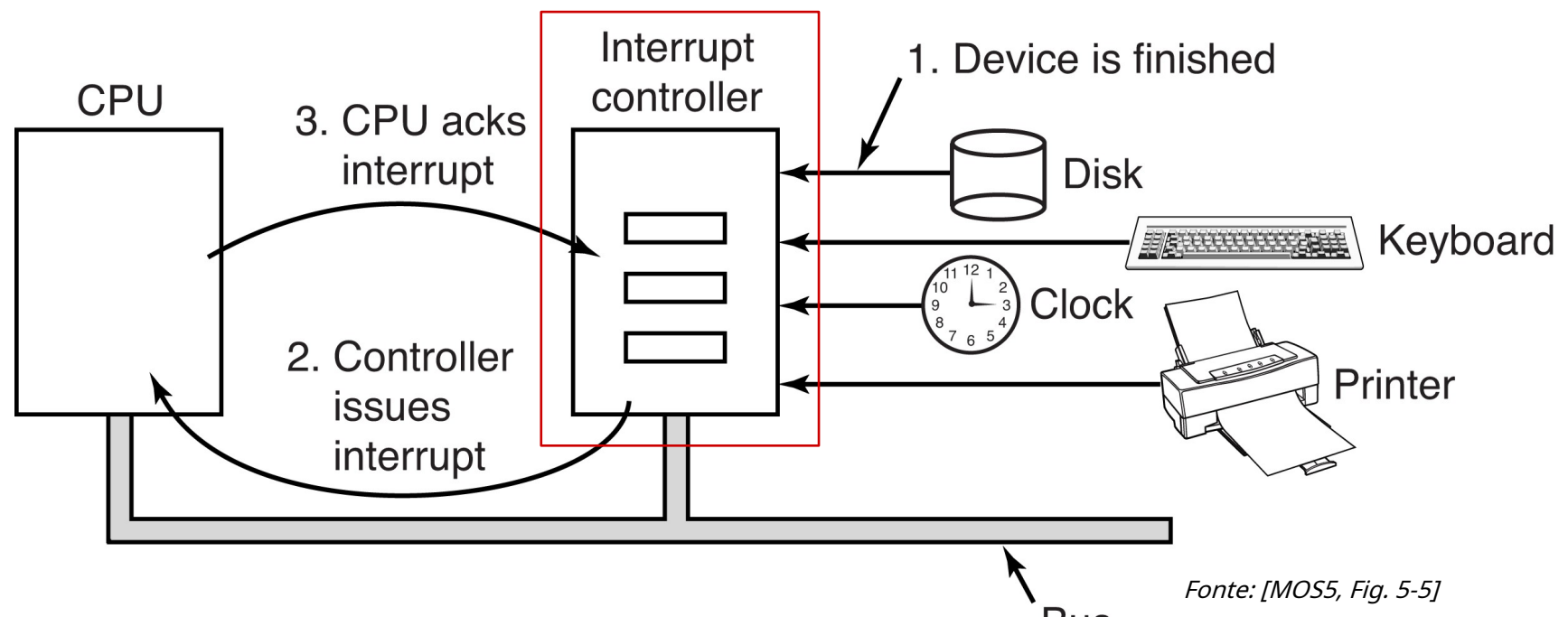
Fonte: [MOS5, Fig. 5-5]

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)

- Il controller degli interrupt ignora l'interrupt se sono pendenti o vengono gestiti interrupt con priorità più alta
 - Il controllore del dispositivo continua a segnalare l'interruzione finché non viene servita

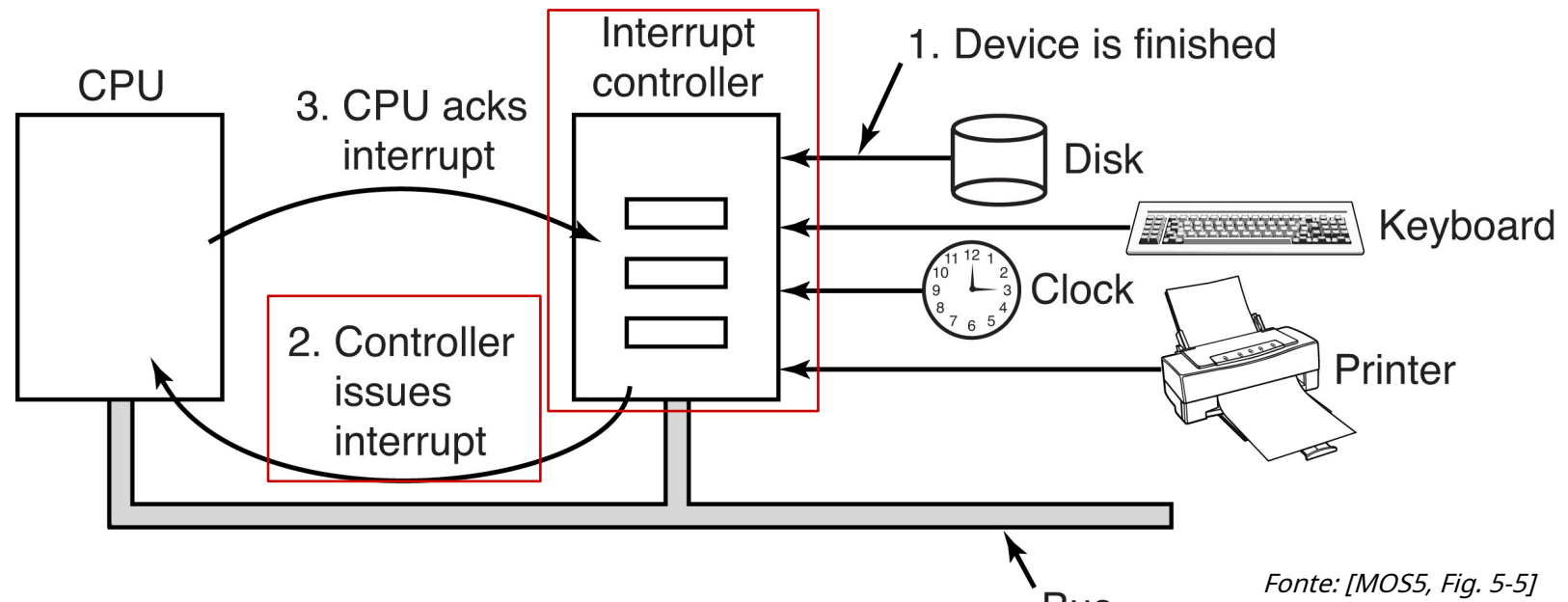


Fonte: [MOS5, Fig. 5-5]

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)
 - Quando è pronto, il controller dell'interruzione rileva l'interruzione
 - Mette *anumeros* sulle righe dell'indirizzo del bus specificando quale dispositivo richiede attenzione
 - Assume un segnale per interrompere la CPU



Fonte: [MOS5, Fig. 5-5]

Hardware I/O: Interazione CPU – Dispositivi

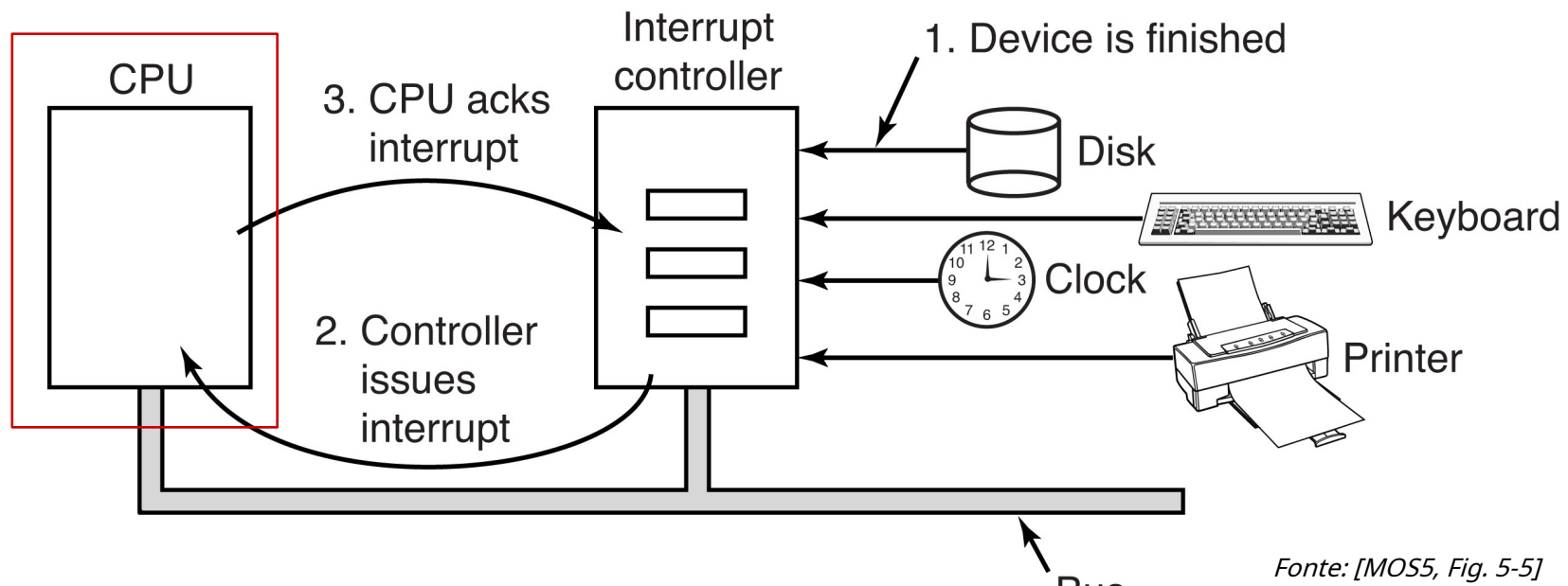
(continua)

- **I/O basato su interrupt**(continua)
 - Il numero sulle linee dell'indirizzo del bus è un indice in una tabella di memoria chiamata **vettore di interruzione**
 - Ogni cella contiene l'indirizzo di memoria che punta all'inizio di un **gestore delle interruzioni**(**procedura di interruzione del servizio**)
 - Una subroutine in grado di gestire gli interrupt da uno specifico dispositivo I/O

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)
 - Dopo che il controller dell'interruzione prende l'interruzione, la CPU inserisce lo stato del processo interrotto (PC, PSW, ...) nello stack corrente e passa alla modalità kernel per servire l'interruzione

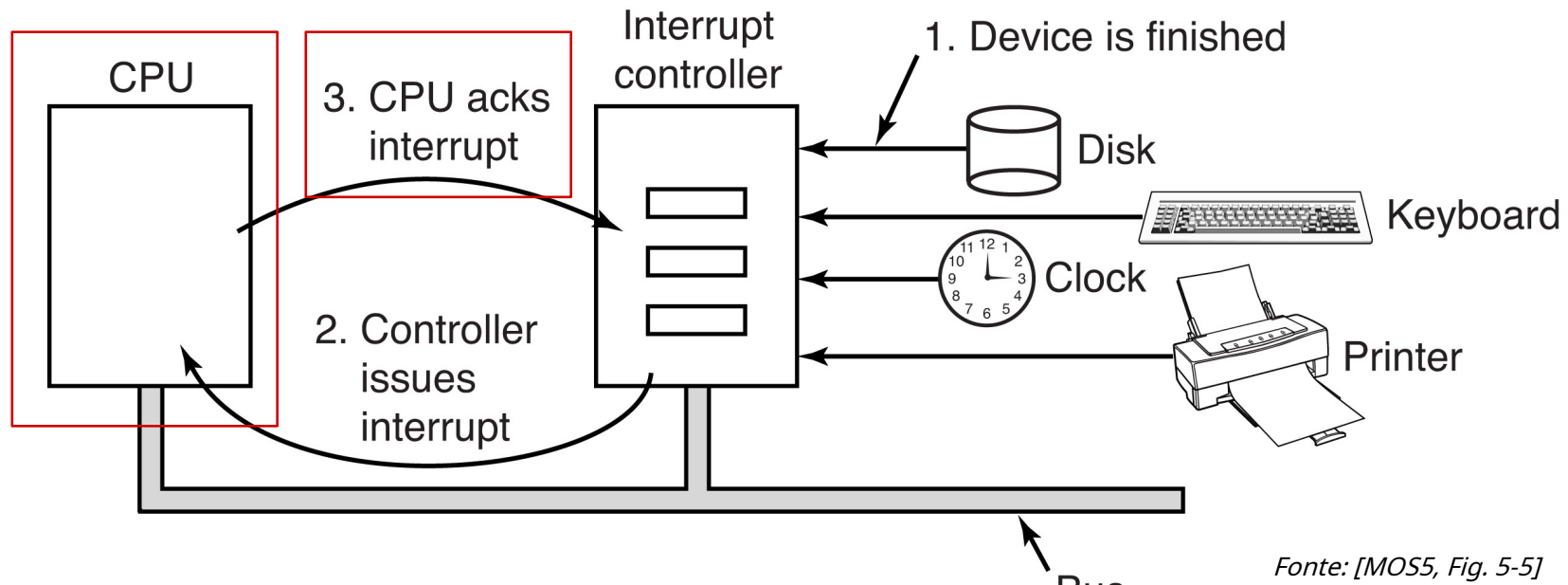


Fonte: [MOS5, Fig. 5-5]

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)
 - La CPU carica il nuovo PC dal vettore di interruzione
 - Il gestore degli interrupt inizia a funzionare
 - La CPU "risponde" l'interruzione al controller dell'interruzione
 - La CPU è pronta a gestire un nuovo interrupt



Fonte: [MOS5, Fig. 5-5]

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O basato su interrupt**(continua)
 - Una volta terminato il gestore delle interruzioni, esegue *arritorno dall'interruzione*istruzione per cancellare l'interruzione
 - La CPU estrae dallo stack lo stato del processo precedentemente in esecuzione (interrotto) per riprenderne l'esecuzione

Hardware I/O: Interazione CPU – Dispositivi

(continua)

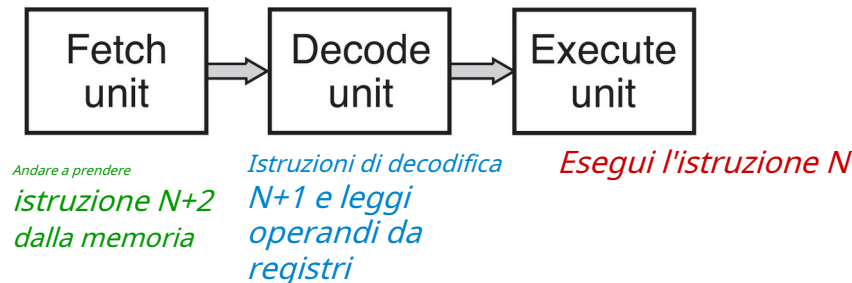
- **I/O basato su interrupt**(continua)
 - Presupposto implicito
 - Quando si verifica un'interruzione subito dopo un'istruzione, *tutte le istruzioni fino a quella inclusa sono state eseguite completamente e nessuna istruzione successiva è stata eseguita affatto*
 - È vero, per i vecchi sistemi informatici
 - La CPU esegue un'istruzione alla volta
 - La CPU (1) recupera l'istruzione successiva dalla memoria, (2) la decodifica per determinarne il tipo e gli operandi e (3) la esegue
 - Dopo aver eseguito un'istruzione, la CPU verifica la presenza di un interrupt in sospeso
 - Falso, per i moderni sistemi informatici
 - Le CPU sono pesantemente **pipelinee** spesso **superscalare**

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)
 - **CPU in pipeline** può eseguire più istruzioni alla volta

- Un'istruzione per ciascuna unità della pipeline



(a)

Fonte: [MOS5, Fig. 1-7]

- Quando si verifica un'interruzione
 - Il PC potrebbe non riflettere il confine corretto tra istruzioni eseguite e non eseguite
 - Molte istruzioni potrebbero essere state eseguite parzialmente

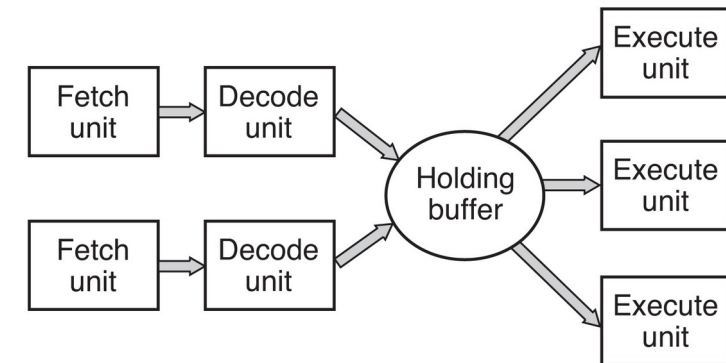
Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)

- **Superscalare** Le CPU sono come CPU pipeline ma in più hanno più unità di esecuzione specializzate (ad esempio, unità aritmetica intera, unità aritmetica in virgola mobile, unità operativa booleana)

- Le istruzioni possono essere scomposte in micro-operazioni
- Le microoperazioni sono assegnate alle varie unità
- Le microoperazioni possono essere eseguite fuori ordine



(b) Fonte: [MOS5, Fig. 1-7]

- Quando si verifica un'interruzione

- Alcune “vecchie” istruzioni potrebbero non essere state elaborate
- Altri “nuovi” potrebbero essere quasi finiti

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **I/O basato su interrupt**(continua)
 - **Interruzione precisa** è un'interruzione che lascia la macchina in uno stato ben definito:
 1. Il PC viene salvato in un luogo noto
 2. Tutte le istruzioni precedenti a quella indicata dal PC sono state completate
 3. Nessuna istruzione oltre a quella indicata dal PC è stata completata
 4. Lo stato di esecuzione dell'istruzione puntata dal PC è noto

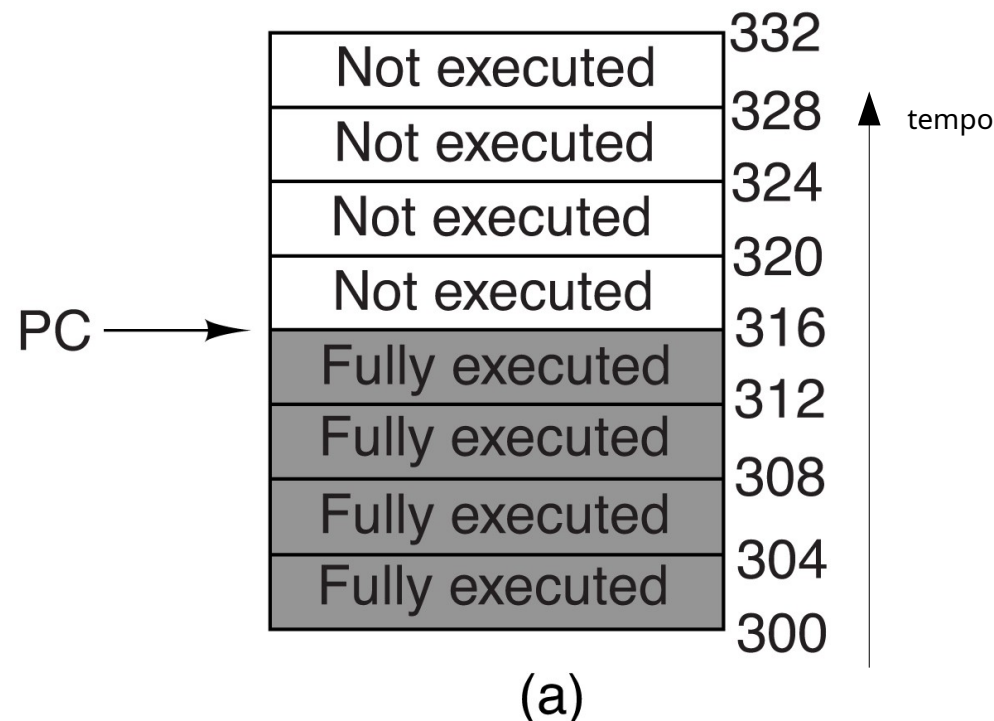
Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O basato su interrupt**(continua)
 - Con interruzioni precise
 - Non esiste alcun divieto di avvio di istruzioni oltre a quella indicata dal PC
 - Qualsiasi modifica apportata ai registri o alla memoria deve essere annullata prima che si verifichi l'interruzione
 - È consentito che l'istruzione puntata dal PC sia stata eseguita o meno

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO:** Interruzione precisa
 - Tutte le istruzioni fino al PC (316) sono state completate e nessuna di quelle successive è stata avviata (o è stata ripristinata per annullarne gli effetti)



Hardware I/O: Interazione CPU – Dispositivi

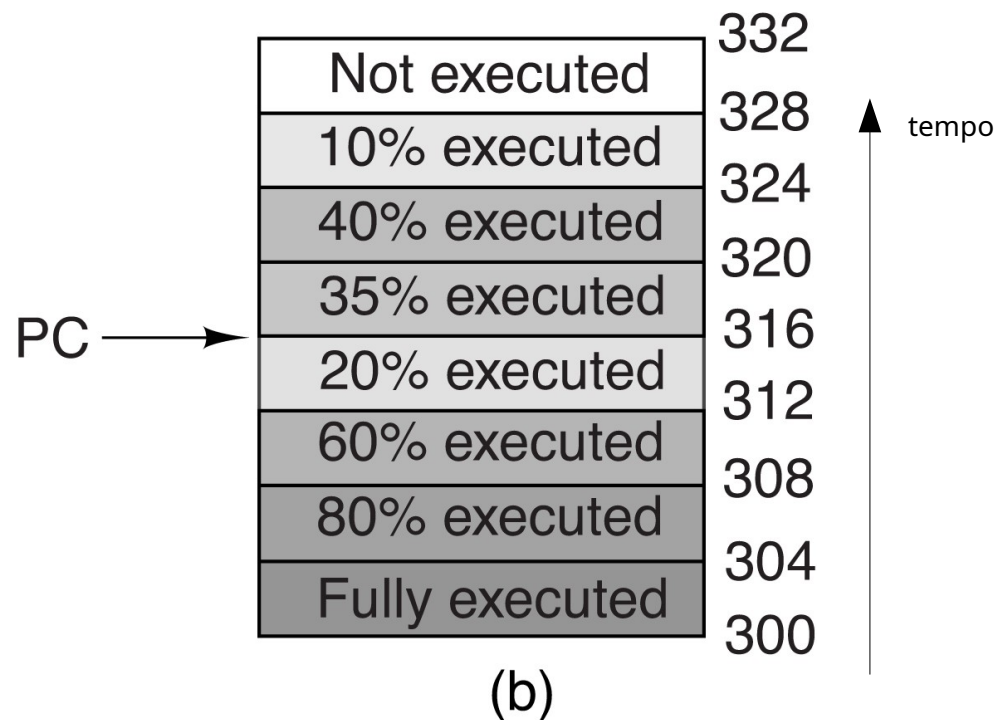
(continua)

- **I/O basato su interrupt**(continua)
 - **Interruzione imprecisa** è un'interruzione che non lo è *preciso*
 - Almeno una delle quattro proprietà è violata
 - Il sistema operativo deve capire cosa è già successo e cosa deve ancora accadere
 - Ripristinare l'esecuzione in HW è molto difficile
 - Le macchine con interruzioni imprecise solitamente salvano una grande quantità di stato interno nello stack
 - In modo che il sistema operativo sia in grado di capire cosa stava succedendo
 - Il codice necessario per ripristinare l'esecuzione dopo un'interruzione imprecisa è tipicamente molto complicato
 - Il salvataggio e il ripristino dello stato su ogni interruzione richiedono molto tempo
 - Non adatto per sistemi con vincoli in tempo reale

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO:** Interruzione imprecisa
 - Diverse istruzioni vicino al PC (316) sono in diverse fasi di completamento, con quelle più vecchie non necessariamente più complete di quelle più giovani

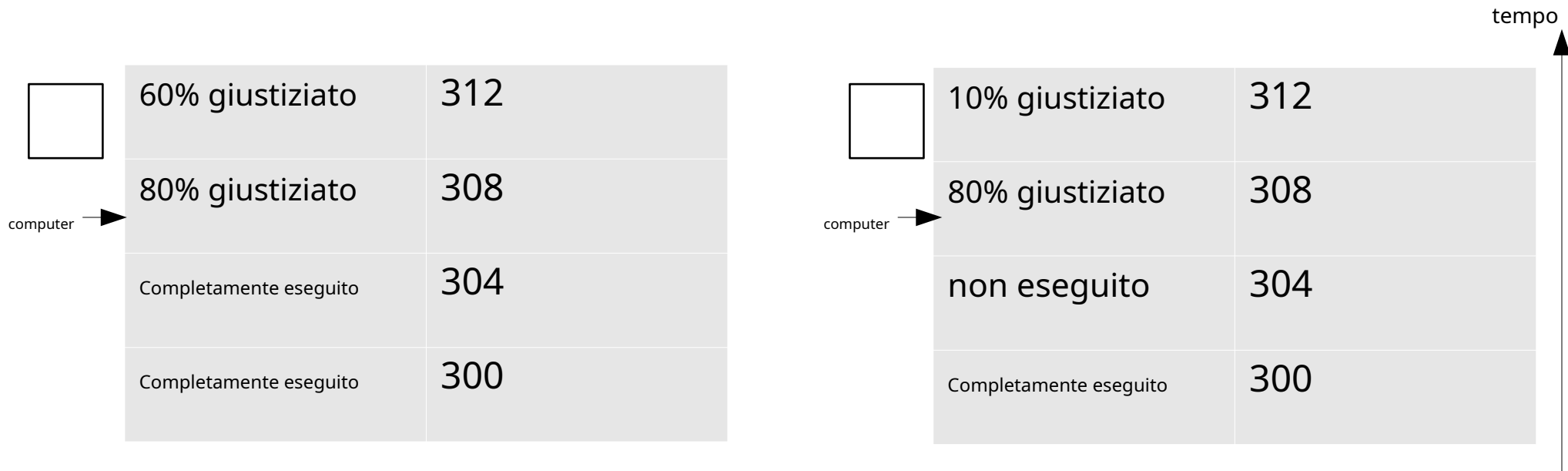


Hardware I/O: Interazione CPU – Dispositivi (continua)

- **I/O basato su interrupt**(continua)
 - Alcuni sistemi hanno entrambi i tipi di interruzioni
 - Per esempio
 - Interruzioni I/O e trap normali → precisi
 - Trappole generate da errori fatali → imprecisi
 - Facile da gestire perché il processo che ha generato la trappola solitamente non viene riavviato

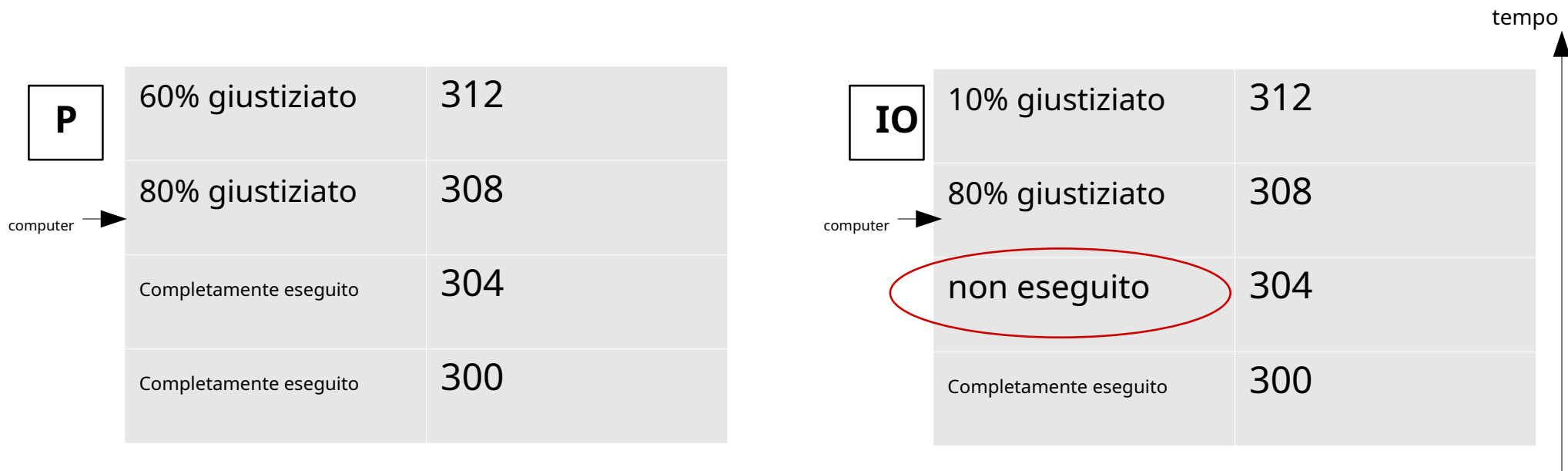
Hardware I/O: Interazione CPU – Dispositivi (continua)

- **ESEMPIO:** Interrupt preciso o impreciso?
 - Considera le seguenti situazioni e contrassegna ciascuna di esse come precisa (P) o imprecisa (I)



Hardware I/O: Interazione CPU – Dispositivi (continua)

- **ESEMPIO:** Interrupt preciso o impreciso? (continua)
 - Considera le seguenti situazioni e contrassegna ciascuna di esse come precisa (P) o imprecisa (I)



Hardware I/O: Interazione CPU – Dispositivi

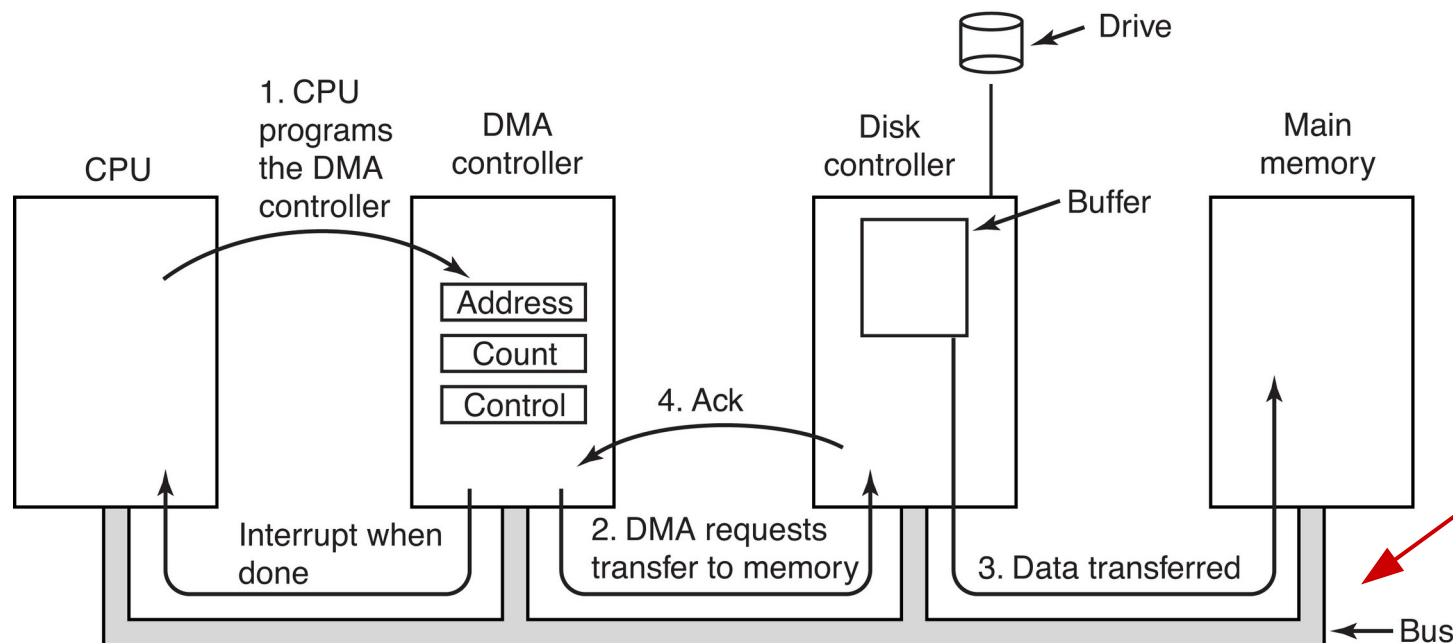
(continua)

- **Accesso diretto alla memoria (DMA)**
 - La CPU scarica parte del lavoro su un processore speciale chiamato **controllore DMA**
 1. La CPU programma il dispositivo I/O e configura il controller DMA; dopodiché può svolgere qualche altro lavoro utile
 2. Il controller DMA effettua trasferimenti di dati tra la memoria e il dispositivo I/O utilizzando PIO
 3. Il controller DMA invia un'interruzione alla CPU al termine
 - Punti di forza
 - Nessuna perdita di tempo della CPU
 - Buono per trasferimenti di dati di grandi dimensioni
 - Debolezza
 - Richiede dispositivi supportati da DMA
 - Complessità extra per gestire il controller DMA
 - Alcuni cicli della CPU vengono spesi per configurare il controller DMA

Hardware I/O: Interazione CPU – Dispositivi

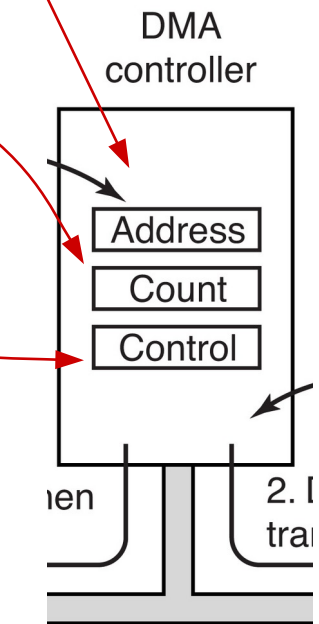
(continua)

- **Accesso diretto alla memoria (DMA)**_(continua)
 - Per semplicità, assumiamo che il controller DMA e la CPU accedano al bus di sistema in modo indipendente



Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - Il controller DMA contiene diversi registri che possono essere scritti e letti dalla CPU
 - **Registro degli indirizzi di memoria**
 - Dove leggere i dati/dove scrivere i dati
 - **Registro del conteggio dei byte**
 - Numero di byte trasferiti finora (per le letture) o da trasferire (per le scritture)
 - **Uno o più registri di controllo**
 - Registro I/O da utilizzare
 - Direzione del trasferimento (lettura/scrittura)
 - Unità di trasferimento (byte/parola)
 - Numero di byte da trasferire in un burst



Fonte: [MOS5, Fig. 5-4]

Hardware I/O: Interazione CPU – Dispositivi

(continua)

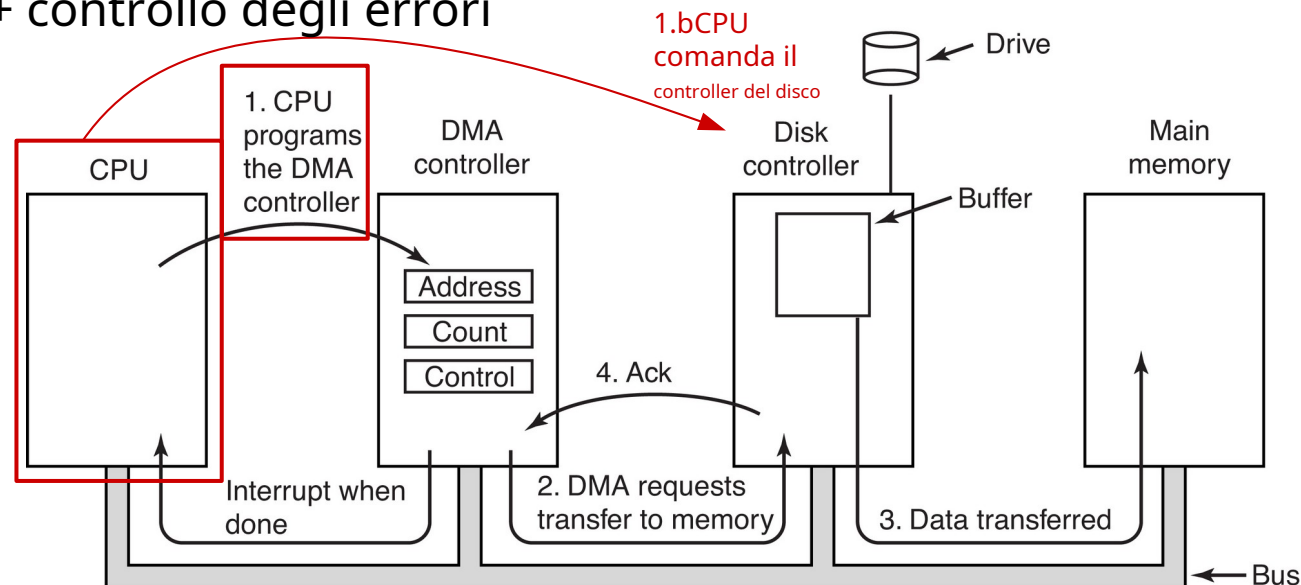
- **ESEMPIO: Lettura da disco con DMA**

1. La CPU programma i registri del controller DMA

- Cosa trasferire dove

1b. CPU invia un comando di lettura al controller del disco

- Il controller del disco inizia a leggere i dati dal disco nel buffer interno + controllo degli errori



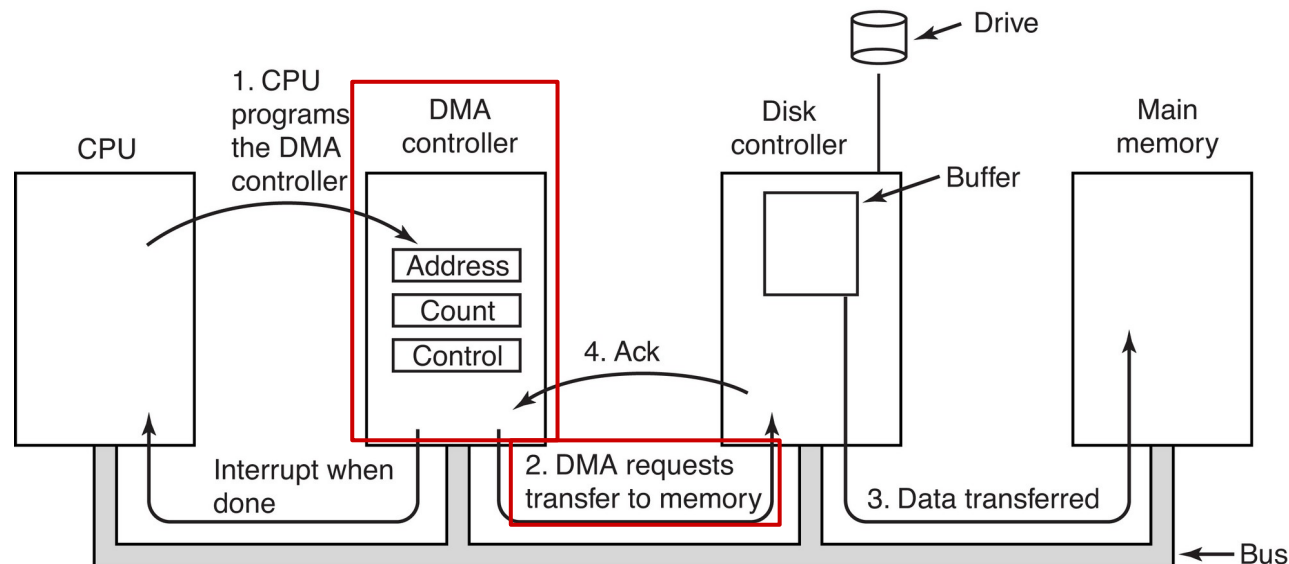
Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO: Lettura da disco con DMA** (continua)

2. Quando sono presenti dati validi nel buffer del controller del disco, inizia il trasferimento DMA

- Il controller DMA inserisce l'indirizzo di memoria di destinazione sul bus
- Il controller DMA invia una richiesta di lettura sul bus al controller del disco



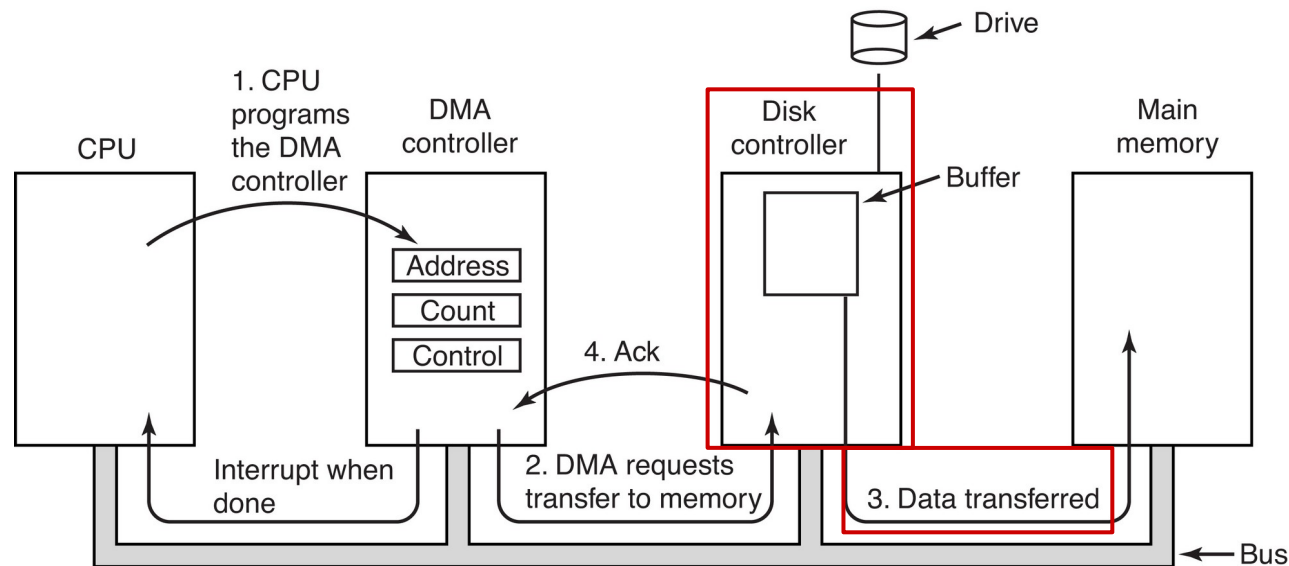
Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO: Lettura da disco con DMA** (continua)

3. Il controller del disco serve la richiesta di lettura

- Il controller del disco preleva una parola dal buffer interno e la scrive nella memoria principale all'indirizzo che trova sul bus



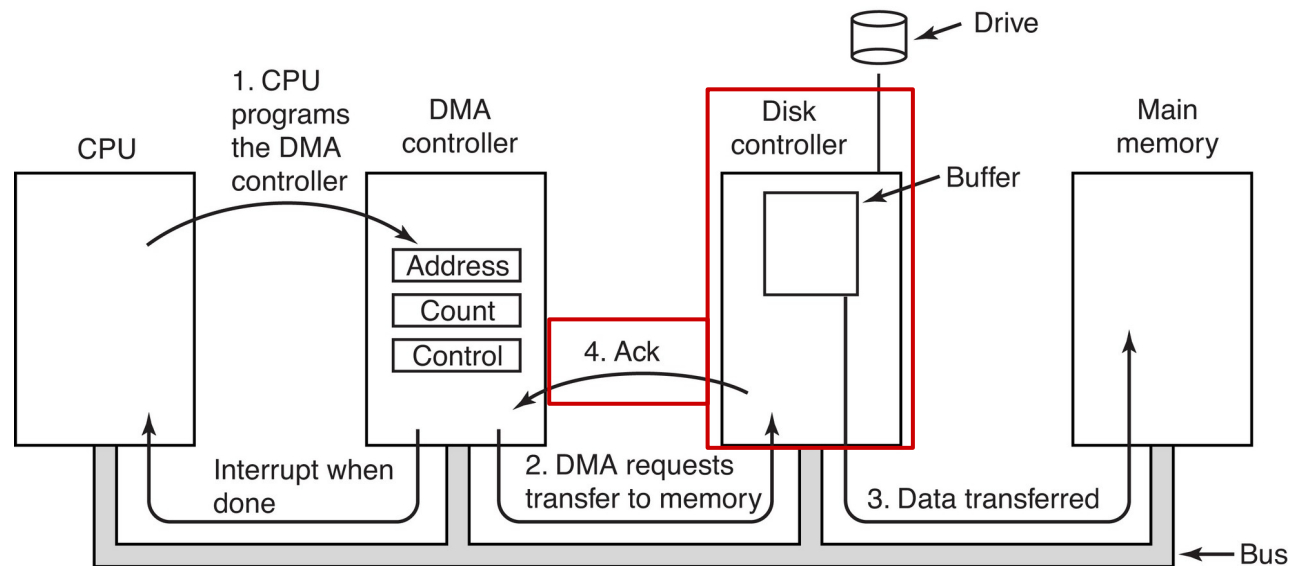
Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO: Lettura da disco con DMA** (continua)

4. Il controller del disco serve la richiesta di lettura

- Una volta terminato il trasferimento in memoria, il controller del disco invia un segnale di riconoscimento al controller DMA attraverso il bus



Hardware I/O: Interazione CPU – Dispositivi

(continua)

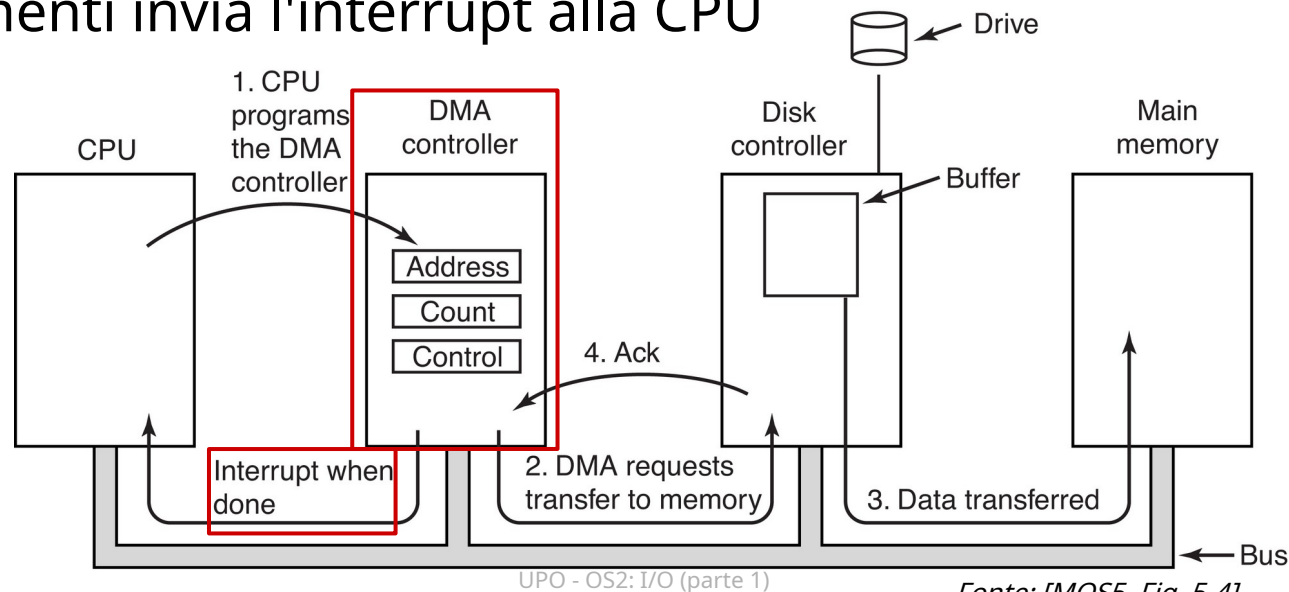
- **ESEMPIO: Lettura da disco con DMA** (continua)

5. Il controller DMA diminuisce il registro del conteggio dei byte

6. Se il conteggio dei byte > 0

- Incrementa il registro degli indirizzi di memoria
- Ripete i passaggi da 2 a 5

7. Altrimenti invia l'interrupt alla CPU

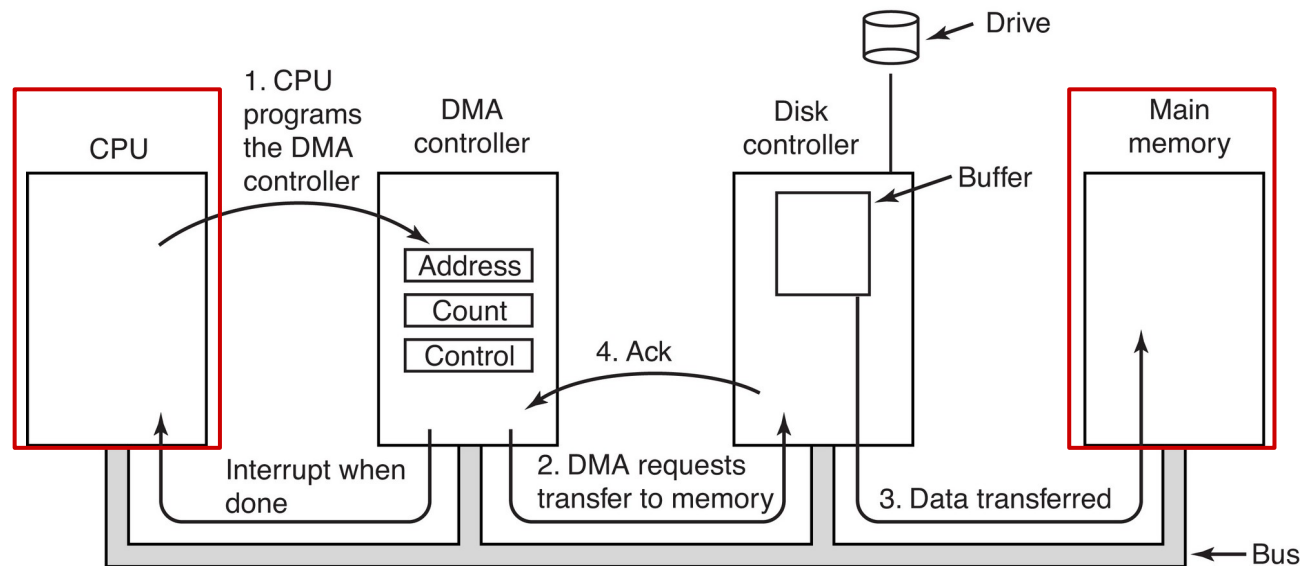


Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **ESEMPIO:** Lettura da disco con DMA (continua)

8. Quando la CPU serve l'interrupt, i dati sono già disponibili nella memoria principale



Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Accesso diretto alla memoria (DMA)**_(continua)
 - Lo schema di trasferimento DMA appena descritto si chiama **modalità volo**
 - I trasferimenti di dati non passano attraverso il controller DMA
 - Ogni trasferimento di dati viene eseguito direttamente dal controller del dispositivo
 - Il controller DMA comunica semplicemente al controller del dispositivo quale operazione eseguire e l'indirizzo di memoria dove leggere/scrivere i dati
 - Punti di forza
 - Efficiente
 - Richiede un solo ciclo di bus per ogni trasferimento dati
 - Punti deboli
 - Un po' limitato
 - Non supporta i trasferimenti di dati diretti da dispositivo a dispositivo e da memoria a memoria

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - Approccio alternativo: **modalità a flusso continuo**
 - I trasferimenti di dati passano attraverso il controller DMA
 - I dati trasferiti vengono prima letti dal dispositivo o dalla memoria in un buffer interno al controller DMA
 - I dati vengono quindi scritti nella memoria o nel dispositivo
 - Punti di forza
 - Più flessibile della modalità fly-by
 - Supporta trasferimenti dati diretti da dispositivo a dispositivo e da memoria a memoria
 - Punti deboli
 - Meno efficiente della modalità fly-by
 - Richiede due cicli del bus: uno per trasferire i dati all'interno del controller DMA e un altro per trasferire i dati al dispositivo/memoria

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - Il controller DMA e la CPU si contendono l'accesso alla memoria in un modo che dipende da come funziona il controller DMA
 - Due approcci:
 - **Furto di biciclette**
 - **Modalità burst**

Hardware I/O: Interazione CPU – Dispositivi

(continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - **Furto di biciclette**approccio
 - Per ogni parola da trasferire, il controller DMA deve prima acquisire il bus e quindi rilasciarlo al termine del trasferimento
 - Il controller DMA ha sempre una priorità del bus più alta rispetto alla CPU (e quindi l'accesso al bus è sempre garantito) perché i dispositivi I/O spesso non possono tollerare ritardi
 - Questo è l'approccio fin qui descritto
 - Punti di forza
 - Impedisce alla CPU di rimanere inattiva per un lungo periodo di tempo
 - Punti deboli
 - Inefficiente
 - Velocità di trasferimento lenta
 - Il controller DMA può occasionalmente rubare un ciclo di bus alla CPU
 - Se anche la CPU vuole accedere al bus deve attendere, ritardando leggermente

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - **Modalità burst**approccio
 - Il controller DMA dice al dispositivo di acquisire il bus, emettere una serie di trasferimenti, quindi rilasciare il bus
 - Punti di forza
 - Più efficiente: i dati possono essere trasferiti con un'unica acquisizione del bus
 - Punti deboli
 - Può bloccare la CPU e altri dispositivi per un periodo prolungato se viene trasferito un burst lungo o se il dispositivo I/O è molto lento

Hardware I/O: Interazione CPU – Dispositivi (continua)

- **Accesso diretto alla memoria (DMA)**(continua)
 - I controller DMA reali possono gestire più trasferimenti di dati contemporaneamente
 - Ogni trasferimento deve coinvolgere un controller del dispositivo diverso
 - Un set di registri, uno per ciascun canale
 - La CPU avvia il trasferimento scrivendo nell'insieme di registri relativi al canale
 - Il controller DMA decide l'ordine di servire ciascun trasferimento
 - Ad esempio, politica round-robin, politica basata sulle priorità,...
 - È possibile che siano in sospeso più richieste a controller di dispositivi diversi contemporaneamente
 - È possibile utilizzare diverse linee di riconoscimento sul bus per ciascun canale DMA per evitare ambiguità nella determinazione del dispositivo

Riferimenti

- **[MOS5]**AS Tanenbaum e H. Bos, "*Sistemi operativi moderni, 5/E*", Pearson, 2023.
- **[OSC9]**A. Silberschatz, PB Galvin e G. Gagne, "*Concetti del sistema operativo, 9/E*", Wiley, 2013.
- **[OSTEP]**RH Arpaci-Dusseau e AC Arpaci-Dusseau, "*Sistemi operativi: tre pezzi facili*", Libri Arpaci-Dusseau, 2015.